



MASTER'S DEGREE PROJECT IN MATHEMATICS

**Pricing and Hedging of Financial Instruments using
Forward–Backward Stochastic Differential Equations**
Call Spread Options with Different Interest Rates for Borrowing and Lending

by

Abigail Hailu Berta

Master's Degree Project in Mathematics

DIVISION OF MATHEMATICS AND PHYSICS
MÄLARDALEN UNIVERSITY
SE-721 23 VÄSTERÅS, SWEDEN



Master's Degree Project in Mathematics

Date of presentation:
27 September 2022

Project name:
Pricing and Hedging of Financial Instruments using Forward–Backward Stochastic Differential Equations (Call Spread Options with Different Interest Rates for Borrowing and Lending)

Author(s):
Abigail Hailu Berta

Version:
3rd October 2022

Supervisor(s):
Achref Bachouch and Marko Dimitrov

Examiner:
Anatoliy Malyarenko (Professor)

Comprising:
30 ECTS credits

Abstract

In this project, we are aiming to solve option pricing and hedging problems numerically via Backward Stochastic Differential Equations (BSDEs). We use Markovian BSDEs to formulate nonlinear pricing and hedging problems of both European and American option types. This method of formulation is crucial for pricing financial instruments since it enables consideration of market imperfections and computations in high dimensions. We conduct numerical experiments of the pricing and hedging problems, where there is a higher interest rate for borrowing than lending, using the least squares Monte Carlo and deep neural network methods. Moreover, based on the experiment results, we point out which method to choose over the other depending on the the problem at hand.

Keywords: Backward stochastic differential equations (BSDEs), Markovian BSDEs, Least square Monte Carlo method, Deep BSDE method, Nonlinear option pricing and hedging problems, Pricing and hedging in high dimensions.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Achref Bachouch, for introducing me to the topic and for his generous time in guiding me at all stages of this project. I would also like to thank Marko Dimitrov for his valuable remarks which significantly contributed to improvement of quality of the thesis.

Contents

Acknowledgements	1
1 Background	6
1.1 European option	8
1.2 American option	8
1.3 Literature review	10
1.4 Project description	11
2 Introduction to Backward Stochastic Differential Equations	12
2.1 Description of notations and spaces	12
2.2 Forward stochastic differential equations	13
2.3 Backward stochastic differential equations	13
2.3.1 Existence and uniqueness of solution to BSDEs	14
2.3.2 Reflected BSDEs	16
2.4 Markovian BSDEs	17
3 Numerical Schemes for BSDEs	19
3.1 Time discretization using the Euler scheme	19
3.2 The least squares regression Monte Carlo method	21
3.3 Approximation with artificial neural networks	23
3.3.1 Artificial neural network introduction	24
3.3.2 The deep BSDE-solver	25
4 Implementation of F-BSDEs to Option Pricing and Hedging Problems	28
4.1 Formulation of the European option pricing and hedging problem as a F-BSDE	28
4.2 Formulation of the American option pricing and hedging problem as a F-BSDE	29
4.3 Numerical experiment	30
4.3.1 Presentation and discussion of results	31
5 Conclusion and Future Works	37
Bibliography	37
A Matlab Code for the LSMC Method using Global Polynomial Bases	41

List of Figures

3.1	An artificial neuron diagram	24
3.2	Activation functions	24
3.3	Fully connected network with 1 input layer, 2 hidden layers and 1 output layer.	25
3.4	Architecture of the deep BSDE-solver	27
4.1	Convergence of Y_0 to the expected value as we change the number of iteration M , time steps N , and the hypercube edges δ for $\alpha_M = 1, \dots, 5$	34
4.2	Convergence of Y_0 to the expected value as we change the number of iteration M , time steps N , and the hypercube edges δ for $\alpha_M = 1, 2$	34

List of Tables

4.1	European vanilla call option (price, standard deviation)	31
4.2	European call spread using the Black–Scholes equation	32
4.3	European call spread using global polynomial basis functions	32
4.4	European call spread using local polynomials of order 0	33
4.5	American call spread using global polynomial basis functions	35
4.6	Mean and standard deviations of the target value and relative errors at different number of iterations using the deep BSDE-solver	36
A.1	European call spread using global polynomial bases (The first 3 Laguerre polynomials)	47
A.2	European call spread using global polynomial bases (Polynomials of order $0, \dots, 3$)	47
A.3	European call spread using local polynomials of order 0	48
A.4	American call spread using global polynomial bases (The first 3 Laguerre polynomials)	48
A.5	American call spread using global polynomial bases (Polynomials of order $0, \dots, 3$)	48

Abbreviations and Notations

BSDE	Backward Stochastic Differential Equation
DNN	Deep Neural Network
DPE	Dynamic Programming Equation
F-BSDE	Forward-Backward Stochastic Differential Equation
LSMC	Least Square Monte Carlo
PDE	Partial Differential Equation
RBSDE	Reflected Backward Stochastic Differential Equation
SDE	Stochastic Differential Equation
SGD	Stochastic Gradient Descent
$(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$	A complete filtered probability space
$\mathcal{F}_t^W$	Natural filtration generated by $W(t)$
$\mathcal{P}_n$	$\mathcal{F}_t$ -Progressively measurable, $\mathbb{R}^n$ -valued processes on $\Omega \times [0, T]$
$\mathbb{L}_n^2(\mathcal{F}_t)$	$\mathcal{F}_t$ -Measurable, $\mathbb{R}^n$ -valued, random variable η s.t. $\mathbb{E}[\eta ^2] < \infty$
$\mathcal{S}_n^2(0, T)$	Adapted continuous processes $Y \in \mathcal{P}_n$, s.t. $\mathbb{E}[\sup_{t \in [0, T]} Y_t ^2] < \infty$
$\mathcal{H}_n^2(0, T):$	Predictable processes $Z \in \mathcal{P}_n$, s.t. $\mathbb{E} \left[\left(\int_0^T Z_t ^2 dt \right) \right] < \infty$

Chapter 1

Background

One of the classical problems in financial mathematics is the problem of pricing contingent claims. A contingent claim is a random variable that represents a future payoff to the holder of the contract. The purpose of the pricing problem is to determine the payoff at time $t = 0$ that the holder should pay to the writer. The project intends to solve numerically pricing and hedging problems of options via Forward-Backward Stochastic Differential Equation (F-BSDE) formulation. The F-BSDE formulation is important in financial problems since it let us incorporate market imperfections and pricing in higher dimensions.

To formulate the pricing and hedging problem, we suppose that the financial market consists of one risk-free asset such as a bond or a money market account and d risky securities. In this paper, we consider stocks as risky assets since they are one of the examples. The formulation is mostly adopted from [13]. The underlying price processes for the risk-free and the risky assets are usually, including in the famous Black–Scholes model introduced in [5], modeled by the Stochastic Differential Equations (SDE) as given below, respectively:

$$dS_t^0 = S_t^0 r_t dt, \quad S_0^0 = 1 \quad (1.1)$$

$$dS_t^i = S_t^i \left(\mu_t^i dt + \sum_{j=1}^d \sigma_t^{ij} dW_t^j \right), \quad S_0^i = S^i > 0, \quad i = 1, \dots, d, \quad (1.2)$$

where

$S_t^0 := \{S^0(t); t \in [0, T]\}$ — price of the risk-free asset at time t ,

$S_t^i := \{S^i(t); t \in [0, T], i = 1, \dots, d\}$ — price of the i th risky security per share at time t ,

$r_t \geq 0$ — short rate which is assumed to be a predictable and bounded process, $\forall t \in [0, T]$,

$\mu_t := (\mu^1, \dots, \mu^d)$ — the stock appreciation rates, a predictable and bounded process, $\forall t \in [0, T]$,

$\sigma_t := (\sigma^{ij}), i, j = 1, \dots, d$ — the volatility matrix, a predictable and bounded process $\forall t \in [0, T]$,

$W = (W^1, \dots, W^d)$ — a Wiener process on $\mathbb{R}^d$, defined on a probability space $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$,

T — maturity time of the option.

Our goal is to find an arbitrage-free price (initial value) of an investment portfolio and the investment strategy which perfectly replicate the terminal payoff (a liability) ξ , i.e., an initial

capital V_0 and a strategy π_0 such that $\Phi(S_T^i) = \xi$. We start with brief descriptions of conditions that characterize arbitrage-free pricing.

The fair price of a contingent claim can be found if there is an attainable self-financing trading strategy, also called a *replicating portfolio*, that does not admit an arbitrage opportunity, see e.g., in [13, and the reference therein]. To construct such a portfolio, let us denote the amount of wealth V_t to be invested in the i th stock at time t by π_t^i . Then

$$V_t = \sum_{i=0}^d \pi_t^i \quad \text{and} \quad \pi_t^0 = V_t - \sum_{i=1}^d \pi_t^i.$$

Definition 1. A self-financing trading strategy is a pair (V_t, π_t) such that (V_t, π_t) satisfies

$$\begin{aligned} V_t &= \left(\frac{\pi_t^0}{S_t^0} \right) S_t^0 + \sum_{i=1}^d \left(\frac{\pi_t^i}{S_t^i} \right) S_t^i \\ dV &= \left(\frac{V_t - \sum_{i=1}^d \pi_t^i}{S_t^0} \right) dS_t^0 + \sum_{i=1}^d \left(\frac{\pi_t^i}{S_t^i} \right) dS_t^i \\ &= \left(V_t - \sum_{i=1}^d \pi_t^i \right) r dt + \sum_{i=1}^d \pi_t^i (\mu_i dt + \sigma_i dW_t) \\ &= r_t V_t dt + \pi_t (\mu_t - r_t \mathbf{1}) dt + \pi_t \sigma_t dW_t \\ &= r_t V_t dt + \pi_t \sigma_t (dW_t + \lambda_t dt), \quad \int_0^T |\sigma_t \pi_t|^2 dt < +\infty, \quad \mathbb{P} \text{ a.s.}, \end{aligned}$$

where $V_t, t \in [0, T]$ is the market value and $\pi_t = (\pi_t^1, \dots, \pi_t^d), t \in [0, T]$ is the portfolio process, $\lambda_t = \sigma_t^{-1}(\mu_t - r_t \mathbf{1}), t \in [t, T]$ is a risk premium, σ^{-1} inverse of the volatility matrix and $\mathbf{1}$ is vector of ones.

Definition 2. An arbitrage opportunity is the existence of a some self-financing trading strategy $(V_t, \pi_t), t \in [t, T]$ such that:

$$V_0 = 0, \quad V_T \geq 0 \quad \mathbb{P} \text{ a.s.} \quad P(V_T > 0) > 0.$$

Definition 3. We say the market is arbitrage free if there is no self-financing portfolio $(V_t, \pi_t), t \in [t, T]$ admitting arbitrage opportunity.

Definition 4. The market is said to be complete if there is non-empty class of hedging strategies for square-integrable non-negative claim ξ , see e.g., [13].

The fundamental theorem of financial mathematics, see e.g., [31], states that in a market model that is free of arbitrage we can construct an equivalent probability measure $\mathbb{Q} \sim \mathbb{P}$ such that the discounted prices of traded instruments are $\mathbb{Q}$ -local martingales. The martingale density process of $\mathbb{Q}$ w.r.t. $\mathbb{P}$ is given by the Radon-Nikodym derivative:

$$\frac{d\mathbb{Q}}{d\mathbb{P}} \Big| \mathcal{F}_t.$$

Then, the fair price of a claim ξ can be computed by taking expectation of the discounted pay-off under this equivalent martingale measure:

$$V_t(S_t, t) = \mathbb{E}^{\mathbb{Q}} \left[e^{-\int_t^T r_u du} \xi \mid \mathcal{F}_t \right].$$

In the next sections, we discuss the two most common option types.

1.1 European option

The European call (put) option gives the holder of the contract the right but not the obligation to buy (sell) the underlying asset for the strike price $K > 0$ only at maturity $T > 0$.

Let us consider one risky asset $S^i = S$ with the following dynamics:

$$dS_t = \mu S_t dt + \sigma S_t dW_t, \quad t \in [0, T]$$

Then payoff function of the European option is given by:

$$\Phi(S_T) = \begin{cases} \max(S_T - K, 0), & \text{call option} \\ \max(K - S_T, 0), & \text{put option.} \end{cases} \quad (1.3)$$

The price of a European option $V(t, S_t), t \in [0, T]$ with terminal condition $\Phi(S_T)$ solves the Black–Scholes partial differential equation (PDE):

$$\begin{cases} \frac{\partial V}{\partial t} + \mathcal{L}_{BS}(V(t, S_t)) - rV(t, S_t) = 0, & (t, S_t) \in [0, T) \times [0, \infty) \\ V(T, S_T) = \Phi(S_T), & S_T \in [0, \infty), \end{cases}$$

where $\mathcal{L}_{BS}$ is the linear Black–Scholes differential operator

$$\mathcal{L}_{BS} = rS \frac{\partial}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2}{\partial S^2}.$$

Under the arbitrage-free argument discussed above the fair price is given by the following Feynman–Kac representation:

$$V(S_t, t) = \mathbb{E}^{\mathbb{Q}} \left[e^{-\int_t^T r_u du} \Phi(S_T) \mid \mathcal{F}_t \right].$$

1.2 American option

The holder of an American option has the freedom to exercise at any time before maturity, to hold the option until maturity or not to exercise at all. This possibility of choosing a favourable time to exercise the option introduces a constraint condition to the value of the American option. The value of this type of option is at least as large as the option payoff g :

$$V(t, S_t) \geq g(S_t), \quad t \in [0, T].$$

The pricing problem for this type of option is formulated as an optimal stopping problem, e.g., in [13], [7], and [3]. Given a continuous payoff function $g: (0, \infty)^d \mapsto \mathbb{R}$, with polynomial growth, the price of the American option is given by:

$$V(t, S_t) = \sup_{\tau \in \mathcal{T}_{[t, T]}} \mathbb{E} \left[e^{-\int_t^\tau r_u du} g(S_\tau) \mid \mathcal{F}_t \right], \quad (1.4)$$

where $V(t, S_t)$ is the value function of an optimal stopping problem given that the option has not been exercised before t , $\mathcal{T}_{[t, T]}$ is collection of $[t, T]$ -valued stopping times. This representation of the process V is called the Snell envelope of g . The payoff function has the form:

$$g(S_t) = \begin{cases} \max(S_t - K, 0), & \text{call option} \\ \max(K - S_t, 0), & \text{put option.} \end{cases}$$

The constraint $V(t, S_t) \geq g(S_t)$, and the fact the option should be exercised only when $V(t, S_t) = g(S_t)$ divides the space into two regions:

- the continuation region

$$C = \{(t, S_t) \in [0, T) \times (0, \infty) : V(t, S_t) > g(S_t)\}, \text{ and}$$

- the stopping (or the exercise) region

$$S = \{(t, S_t) \in [0, T) \times (0, \infty) : V(t, S_t) \leq g(S_t)\}.$$

The value of American option solves the linear complementary problem, also known as the quasi-variational formulation e.g., in [3, see the reference therein]:

$$\begin{cases} \min \left[rV(t, S_t) - \frac{\partial V}{\partial t} - \mathcal{L}_{BS}(V(t, S_t), V(t, S_t)) - g(S_t) \right] = 0, & (t, S_t) \in [0, T) \times [0, \infty) \\ V(T, S_T) = g(S_T), & S_T \in [0, \infty). \end{cases}$$

The Black–Scholes PDE is a linear parabolic PDE thanks to the model assumptions, e.g., no transaction costs are involved, short selling is allowed, a constant short-term interest rate is assumed for both borrowing and lending.

However, suppose we want to incorporate incomplete market conditions, such as when a different interest rate for borrowing lending is considered, when transaction costs are involved, or when short selling is not allowed. In that case, the PDE will no longer become linear. In such cases, getting the risk-free probability measure and analytical formulae becomes complicated or impossible. Hence the linear Feynman–Kac representation does not apply to these cases.

The authors in [13] have proven that a pair of processes $(V_t, \pi_t \sigma_t)$, $t \in [0, T]$ representing the fair price and a hedging strategy can be found under the objective probability measure $\mathbb{P}$ using a F-BSDE formulation for a square integrable $\sigma_t \pi_t$, $t \in [0, T]$. The American type option can also be solved using F-BSDEs, see e.g., [12], by introducing additional processes for the barrier and for the time value. The Backward Stochastic Differential Equation (BSDE) with these additional features is called *Reflected* BSDEs (RBSDEs).

1.3 Literature review

In this project we will be considering the financial applications of F-BSDEs of the form:

$$\begin{cases} X_t = x_0 + \int_0^t b(s, X_s)ds + \int_0^t \sigma(X_s)dW_s, & 0 \leq t \leq T \\ Y_t = \xi + \int_t^T f(s, X_s, Y_s, Z_s)ds - \int_t^T Z_s dW_s, \end{cases}$$

where X is a forward process, Y is backward process, Z is a *control* processes, W is a Wiener process, ξ a random terminal condition of Y and f is a given function called a *generator* satisfying regularity conditions discussed e.g., in [27] and [13].

BSDEs received considerable attention for a wide range of applications such as stochastic control problems [11] and derivative pricing [13, 12] after the existence and uniqueness results had been proven, for a Lipschitz continuous generator function, by Pardoux and Peng [26] in 1990. Authors of [27] showed that the solutions of F-BSDEs, where the randomness of the backward processes comes from the forward processes, can be expressed as deterministic functions of time and the state processes. The F-BSDEs are also called Markovian BSDEs. Pardoux and S. Tang [27] have further shown that the solutions can be solutions to a semilinear parabolic PDEs under some regularity conditions. This relation between F-BSDEs and semilinear parabolic PDEs is considered to be important for many applications.

F-BSDEs are particularly important for pricing financial derivatives because we can incorporate certain market imperfections. In [13] hedging claims with higher interest rates for borrowing and hedging claims in incomplete markets, i.e., when only some primary securities can be traded, are illustrated. The presence of transaction costs can also be considered in the BSDE formulation, see e.g., [29]. Furthermore, F-BSDEs have a significant role when we have several assets in the portfolio, i.e., in the case of high-dimensional pricing problems. The fact that we do not need to convert the objective probability measure to a risk neutral measure can also be considered as an additional advantage.

The solutions to the BSDEs involve expectations, see e.g., [30]. To solve numerically, we need first a time discretization procedure; the Euler scheme is the most popular one, see e.g., in [30] and [6]. Then numerical approximation procedures for the expectations follows. The approximation methods include Malliavin calculus based method [6], quantization methods [1], the Least Squares Monte Carlo (LSMC) method [19, 23], and the Deep Neural Network (DNN) method [10, 21]. Among these we cover two of them, namely the classical LSMC method and the DNN method, in this project.

The LSMC method was proposed by [19, 23]. In this approach the problem is solved backward in time. Then nested conditional expectations are evaluated backward through the time steps by replacing the conditional expectations by some estimator function. Although the method is quite effective for low dimensions, say up to $d = 3$, it suffers from the *curse of dimensionality* as the number of dimensions increase, see e.g., [18, 10, references therein]. On the other hand, high dimensions have significant practical relevance in mathematical finance, where the spatial dimension is determined by the number of financial assets in the market.

The other approximation method is the DNN method which has recently become attractive for solving nonlinear PDEs and BSDEs. A feed-forward neural network algorithm was developed by [10, 21]. By taking advantage of the well-known connection between PDEs and

BSDEs, the gradient of the unknown solution, which is related to the control process Z , is approximated by DNNs. The gradient serves as a policy function while the difference between the approximated solution and the known terminal condition plays the role of the loss function.

An essential advantage of employing the DNN method is that the curse of dimensionality can be avoided. The deep BSDE-solver developed by [10] is tested to be efficient for up to 100 dimensions. As a drawback the authors in [10, 21] pointed out that the multilayer neural networks lack a theoretical framework for explaining the effectiveness.

1.4 Project description

The remaining part of the paper is organised as follows. In Chapter 2, brief introductions of the forward SDE, BSDEs in general and the reflected BSDEs and Markovian BSDEs in particular are presented. Additionally, proofs of existence and uniqueness of the solution to the BSDEs and RBSDEs are included. The numerical schemes are discussed in Chapter 3. These included the time discretization and the approximation procedures. The approximation methods specifically are the LSMC and the DNN methods. In Chapter 4, we formulate the linear and a nonlinear pricing problems for both European and American options via the F-BSDEs. Call spread options are also considered in addition to vanilla options. We further include numerical experiments using the classical LSMC approximation method and using a deep BSDE-solver developed by [10]. In the last chapter, we attempt to point out the contribution and limitations of the paper. We also point out potential future works that can be done which we do not manage to do at this time due to time and scope constraints.

Chapter 2

Introduction to Backward Stochastic Differential Equations

In this chapter, we present brief introductions of SDEs, BSDEs, RBSDEs and F-BSDEs. Moreover, proofs of existence and uniqueness of solutions to the BSDEs and RBSDEs are included.

2.1 Description of notations and spaces

We consider a probability space $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$, where Ω is a set of all events ω , $\mathcal{F}$ is a σ -field of measurable events, $\mathbb{F} = \{\mathcal{F}_t, t \in [0, T]\}$ is a family of information sets which satisfy $\mathcal{F}_s \subseteq \mathcal{F}_t$ for $s \leq t$, $\mathbb{P}$ is the real (objective) probability measure, and $T > 0$ is a fixed time horizon. The natural filtration of a process X_t , or the filtration generated by process X_t , is defined as $\mathcal{F}_t^X = \sigma(X_s, 0 \leq s \leq t)$, i.e. information contained in the random variables $X_s, 0 \leq s \leq t$. We consider also a Wiener process W_t defined on the probability space $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ and a filtration generated by the Wiener process $\mathcal{F}_t^W$ but we will simply denote it as $\mathcal{F}_t$ in this paper for brevity.

A process Y is said to be adapted to the filtration $\mathbb{F}$ if Y is $\mathbb{F}$ -measurable, i.e. Y_t is a function of the information contained in $\mathcal{F}_t$ for every $t \in [0, T]$. We present below the formal definitions of adapted, predictable and progressively measurable process.

Definition 5 (Adapted process [31]). A stochastic process $\{X(t); t \in [0, T]\}$ is said to be adapted to the filtration $\mathbb{F}$, if each $X(t)$ is measurable with respect to $\mathcal{F}_t$.

Let $\mathcal{L}$ be a set of all left-continuous $\mathbb{F}$ -adapted stochastic processes.

Definition 6 (Predictable processes [8]). A predictable σ -field, $\mathcal{P}$, is the smallest σ -field of subsets of $[0, T] \times \Omega$ with respect to which all processes from $\mathcal{L}$ are measurable. A stochastic process $X(t)$ is predictable if the function $(t, \omega) \mapsto X(t, \omega)$ is $\mathcal{P}$ measurable.

Definition 7 (Progressively measurable [31]). We say X is $\mathbb{F}$ -progressively measurable, or simply called $\mathbb{F}$ -measurable, if the restriction of X on $[0, t]$ is $\mathcal{B}([0, t]) \times \mathcal{F}_t$ -measurable for any $t \in [0, T]$, where $\mathcal{B}([0, t])$ denotes the Borel sets in $[0, t]$.

Let $|\cdot|$ denote the Euclidean norm, then the process spaces are defined as follows:

- $\mathcal{P}_n$: $\mathcal{F}_t$ -progressively measurable, $\mathbb{R}^n$ -valued processes on $\Omega \times [0, T]$,
- $\mathbb{L}_n^2(\mathcal{F}_t)$: $\mathcal{F}_t$ -measurable, $\mathbb{R}^n$ -valued, random variables η s.t. $\mathbb{E}[|\eta|^2] < \infty$,
- $\mathcal{S}_n^2(0, T)$: adapted continuous processes $Y \in \mathcal{P}_n$, s.t. $\mathbb{E}[\sup_{t \in [0, T]} |Y_t|^2] < \infty$,
- $\mathcal{H}_n^2(0, T)$: predictable processes $Z \in \mathcal{P}_n$, s.t. $\mathbb{E}\left[\left(\int_0^T |Z_t|^2 dt\right)\right] < \infty$.

2.2 Forward stochastic differential equations

Forward SDEs, which we simply call SDEs, are ordinary differential equations to which stochastic terms are added. Given an initial condition $X_0 = x_0$, a drift coefficient $b(t, X_t)$ and a diffusion coefficient $\sigma(t, X_t)$ the SDE with a solution denoted by X_t is of the form:

$$X_t = x_0 + \int_0^t b(s, X_s) ds + \int_0^t \sigma(X_s) dW_s, \quad 0 \leq t \leq T. \quad (2.1)$$

The SDE can also be formulated, alternatively, as:

$$\begin{cases} dX_t = b(t, X_t) dt + \sigma(X_t) dW_t \\ X_0 = x_0. \end{cases}$$

For the existence and uniqueness of the solution, the following assumptions are imposed to the drift and diffusion coefficients:

- (F1) the coefficients b and σ are uniformly Lipschitz continuous with respect to x , i.e., $|b(t, x) - b(t, x')| + |\sigma(t, x) - \sigma(t, x')| \leq C_{b, \sigma}(1 + |x - x'|)$,
- (F2) the coefficients satisfy a linear growth condition, i.e. there exists a constant c depending on T and $C_{b, \sigma}$ s.t. for any (t, x) , $|\sigma(t, x)| + |b(t, x)| \leq c(1 + |x|)$.

Theorem 1. *Let the initial condition $x_0 \in \mathbb{L}_n^2(\mathbb{R})$, and coefficients $b(t, X_t): \mathbb{R}^+ \times \mathbb{R}^d \mapsto \mathbb{R}^d$, and $\sigma(t, X_t): \mathbb{R}^+ \times \mathbb{R}^d \mapsto \mathbb{R}^{d \times d}$. Let also the coefficients b , and σ satisfy the assumptions (F1)-(F2) above. Then there exists a unique adapted solution $X_t \in \mathcal{S}_d^2(0, T)$ to the SDE in Eq. (2.1), $\forall t \in [0, T]$.*

The solution under these assumptions is referred to as the *strong solution*. Further discussions and proofs can be found e.g., in [18].

2.3 Backward stochastic differential equations

The BSDE has, on the other hand, a terminal condition specified and we must find the solution at the initial time $t = 0$. Further, the solution to a BSDE consists of pair of process. Given

a terminal condition as an $\mathbb{R}^d$ -valued random variable $Y_T = \xi$ in $\mathbb{L}^2$ and a function called a *generator* or a *driver* $f: [0, T] \times \Omega \times \mathbb{R}^d \times \mathbb{R}^{d \times d} \mapsto \mathbb{R}^d$, the BSDE is defined as:

$$Y_t = \xi + \int_t^T f(s, Y_s, Z_s) ds - \int_t^T Z_s dW_s \quad 0 \leq t \leq T, \quad (2.2)$$

where T is a fixed time horizon (maturity time) and $t, s \in [0, T]$. Equivalently, Eq. (2.2) can be written in differential form as follows:

$$\begin{cases} -dY_t = f(t, Y_t, Z_t) dt - Z_t dW_t \\ Y_T = \xi. \end{cases}$$

The solution to the problem in Eq. (2.2) is a pair of process $(Y, Z) \in \mathcal{S}_d^2 \times \mathcal{H}_d^2$ adapted and predictable processes, respectively. The random process Z is called a *control process*. It controls the adapted process Y to satisfy the terminal condition. In the next section we discuss the existence and uniqueness of the BSDE solution, taken from [14].

2.3.1 Existence and uniqueness of solution to BSDEs

The BSDE in Eq. (2.2) does not have a unique solution in general. However, under some specific assumptions on the generator function f , it can be proved that it has a unique solution. Pardoux and Peng [26] proved the existence and uniqueness of a solution to the BSDE in Eq. 2.2 when the generator function f is Lipschitz continuous with respect to (y, z) and $\xi_T \in \mathbb{L}^2(\Omega)$ is a square integrable random variable.

Assumptions that are considered as standards, e.g., in [14] are the following:

(B1) the terminal condition $\xi \in \mathbb{L}^2(\mathbb{R})$,

(B2) $(f(t, 0, 0))_{t \leq T} \in \mathcal{H}_d^2$,

(B3) f is uniformly Lipschitz w.r.t. (y, z) : there exists a constant $C \geq 0$ s.t.

$$\forall (y, y', z, z') \quad |f(\omega, t, y, z) - f(\omega, t, y', z')| \leq C|y - y'| + |z - z'|, \quad dt \times d\mathbb{P}.$$

Under the standard assumptions listed above, we present a glimpse of the proof of existence and uniqueness of the solution.

Theorem 2 (Existence and uniqueness of solution [14]). *Suppose assumptions (B1)-(B3) hold. Then there exists a unique solution $(Y, Z) \in \mathcal{S}_d^2 \times \mathcal{H}_d^2$ of the BSDE with parameters (f, ξ) .*

Proof. i) When $f \equiv 0$

Let us first consider a BSDE with no generator, and rewrite Eq. (2.2) as:

$$Y_t = \xi - \int_t^T Z_s dW_s. \quad (2.3)$$

Taking the conditional expectation of Eq. (2.3) with respect to $\mathcal{F}_t$ gives:

$$Y_t = \mathbb{E}[\xi | \mathcal{F}_t], \quad t \in [0, T],$$

because Y is $\mathbb{F}$ -measurable and the expectation of increment of the Wiener process is zero. Denote $m_t = \mathbb{E}[\xi|\mathcal{F}_t]$. Using martingale representation theorem, see e.g., [31] which states there exist a unique $Z \in \mathcal{H}_d^2(0, T)$, s.t.

$$\begin{aligned} m_t &= m_0 + \int_0^t Z_s dW_s, & \text{forward representation} \\ &= m_T - \int_t^T Z_s dW_s, & \text{backward representation,} \end{aligned}$$

we get:

$$Y_t = \mathbb{E}[\xi|\mathcal{F}_t] = \mathbb{E}[\xi] + \int_0^t Z_s dW_s.$$

ii) When $f \equiv f(t, \omega, 0, 0) = f(t, \omega)$

Now, we consider the case when the generator does not dependent on the solutions Y and Z .

$$Y_t = \xi + \int_t^T f(s, \omega) ds - \int_t^T Z_s dW_s. \quad (2.4)$$

If we add $\int_0^t f(s, \omega) ds$ to both sides of Eq. (2.4), we get:

$$Y_t + \int_0^t f(s, \omega) ds = \xi + \int_0^T f(s, \omega) ds - \int_t^T Z_s dW_s.$$

Then let

$$\begin{aligned} Y'_t &= Y_t + \int_0^t f(s, \omega) ds \\ \xi' &= \xi + \int_0^T f(s, \omega) ds. \end{aligned}$$

Since $f(t, \omega)$ is independent of Y and Z , the BSDE we have now is equivalent to a BSDE with zero generator and a terminal value ξ' . Therefor the proof can be completed in similar way as in the case with no generator.

iii) When the generator function is Lipschitz continuous

Here the existence is obtained by a *contraction* argument. This is done in [14] by introducing a contraction application $\Gamma : \mathcal{D}_\alpha \mapsto \mathcal{D}_\alpha$, where the space $\mathcal{D}_\alpha = \mathcal{S}_d^2 \times \mathcal{H}_d^2$ with the norm:

$$\|(Y, Z)\|_\alpha = \left(\mathbb{E} \left[e^{\alpha s} (|Y_s|^2 + |Z_s|^2) ds \right] \right)^{\frac{1}{2}}.$$

Define:

$$\begin{aligned} (u_t, v_t) \in \mathcal{D}_\alpha &\mapsto \Gamma(u_t, v_t) := (Y_t^{u,v}, Z_t^{u,v}), \\ Y_t^{u,v} &= \xi + \int_t^T f(s, \omega, u, v) ds - \int_t^T Z_s^{u,v} dW_s. \end{aligned} \quad (2.5)$$

The BSDE in Eq. (2.5) has a generator function which does not depend on Y and Z . Therefore, we can use the argument in ii) to show that Eq. (2.5) has a unique solution $(Y_t^{u,v}, Z_t^{u,v}) \in \mathcal{D}_\alpha$. Finally, we show that Γ is contraction:

$$\begin{aligned} \|\Gamma(u, v) - \Gamma(u', v')\|_\alpha &= \|(Y^{u,v}, Z^{u,v}) - (Y^{u',v'}, Z^{u',v'})\|_\alpha \\ &\leq \kappa \|(u, v) - (u', v')\|_\alpha, \quad \text{for } 0 \leq \kappa < 1. \end{aligned}$$

Detailed proof of this can be found e.g., in [14]. $\square$

2.3.2 Reflected BSDEs

When a constraint is introduced to the solution of BSDEs, an additional control process is added to the dynamics. Let us introduce a continuous process $L \in \mathcal{S}_d^2$ as a lower bound to Y , also called an *obstacle* process. In this case, we have a RBSDE with a terminal condition $\xi \in \mathbb{L}^2$ and a generator f satisfying assumptions A(2)-A(3) listed in Subsection 2.3.1, of the form:

$$\begin{aligned} Y &\in \mathcal{S}_d^2, \quad Z \in \mathcal{H}_d^2, \text{ and } C_T \in \mathbb{L}^2, \\ Y_t &= \xi + \int_t^T f(s, Y_s, Z_s) ds + C_T - C_t - \int_t^T Z_s dW_s \quad 0 \leq t \leq T, \end{aligned} \quad (2.6)$$

$$Y_t \geq L_t, \quad (2.7)$$

$$\{C_t\} \text{ a continuous increasing process, } C_0 = 0, \text{ and } \int_0^T (Y_t - L_t) dC_t = 0. \quad (2.8)$$

The solution to a one dimensional RBSDE is a triple of processes taking value in $(Y, Z, C) \in \mathbb{R}^{1+d+1}$. The process C in Eq. (2.6) is added in order to force the process Y to stay above the obstacle L . The process C acts only when Y reaches the obstacle L .

Theorem 3 (Solution to the RBSDE [12]). *Let $\{(Y_t, Z_t, C_t), t \in [0, T]\}$ be a solution of the RBSDE satisfying conditions in Eqs. (2.6)-(2.8); then for each $t \in [0, T]$,*

$$Y_t = \text{ess sup}_{\tau \in \mathcal{T}_{[t, T]}} \mathbb{E} \left[\int_t^\tau f(s, Y_s, Z_s) ds + L_\tau \mathbb{1}_{\{\tau < T\}} + \xi \mathbb{1}_{\{\tau = T\}} \mid \mathcal{F}_t \right], \quad (2.9)$$

where $\mathcal{T}$ is the set of all stopping times dominated by T .

Proof. Take the conditional expectation of Eq. (2.6) between t and τ :

$$\begin{aligned} Y_t &= \mathbb{E} \left[Y_\tau + \int_t^\tau f(s, Y_s, Z_s) ds + C_\tau - C_t \mid \mathcal{F}_t \right] \\ &\geq \mathbb{E} \left[L_\tau \mathbb{1}_{\{\tau < T\}} + \xi \mathbb{1}_{\{\tau = T\}} + \int_t^\tau f(s, Y_s, Z_s) ds \mid \mathcal{F}_t \right], \end{aligned}$$

because C is increasing. Let us define the optimal stopping time:

$$D_t = \inf\{t \leq u \leq T; Y_u = L_u\}.$$

From Eq. (2.8) we notice that $dC_t = 0$ when $Y_t > L_t, t \leq T$. Thus $C_{D_t} - C_t = 0$. This gives the equality:

$$Y_t = \mathbb{E} \left[L_{D_t} \mathbb{1}_{\{D_t < T\}} + \xi \mathbb{1}_{\{D_t = T\}} + \int_t^{D_t} f(s, Y_s, Z_s) ds \mid \mathcal{F}_t \right].$$

□

The existence of the solution is proved in [12] using a fixed point argument and by an approximation via penalization techniques and also in [14] via penalization and via Snell envelop methods.

2.4 Markovian BSDEs

When the randomness of the BSDE parameters (f, ξ) comes from the forward process, we get a system of equations called the F-BSDE. In the F-BSDE the time and space variables (t, x) of the SDE are used as data in the parameters (f, ξ) of the BSDE, i.e., f depends on $X_t, t \in [0, T]$ and the terminal condition of the BSDE depends on X_T . Given x_0 , the functions b, σ , a terminal condition $Y_T = \xi = \Phi(X_T)$ and a generator f the F-BSDE becomes:

$$\begin{aligned} X_t &= x_0 + \int_0^t b(s, X_s) ds + \int_0^t \sigma(s, X_s) dW_s, \quad 0 \leq t \leq T \\ Y_t &= \Phi(X_T) + \int_t^T f(s, X_s, Y_s, Z_s) ds - \int_t^T Z_s dW_s. \end{aligned} \tag{2.10}$$

We call the F-BSDE in Eq. (2.10) a *decoupled* F-BSDE. When the drift term in X depends on Y the F-BSDE is called a *coupled* F-BSDE.

Theorem 4. *Let the forward and backward processes in Eq. (2.10) satisfy the regularity conditions (F1)-(F2) and (B1)-(B3) in Sections 2.2 and 2.3, respectively. Let also the parameters b, σ , and f are uniformly Hölder- $1/2$ continuous in t with Hölder constant $\mathcal{K}$:*

$$|f(t, x, y, z)| + |\Phi(x)| \leq \mathcal{K}(1 + |x|^p), \quad p \geq 1/2.$$

Then, there exists a unique solution to the F-BSDE in Eq. (2.10).

Proof. See [13, 14, 31] for proof and further details. □

Under the regularity conditions assumed for the existence of unique solution to the F-BSDE, we can relate (Y, Z) to the pair (solution, gradient) of semilinear parabolic PDE of the form:

$$\begin{aligned} \frac{\partial u}{\partial t}(t, x) + \frac{1}{2} \text{Tr} \left(\sigma \sigma^\top(t, x) \nabla^2 u(t, x) \right) + \nabla u(t, x) \cdot b(t, x) \\ + f(t, x, u(t, x), \sigma^\top(t, x) \nabla u(t, x)) = 0, \end{aligned} \tag{2.11}$$

with a solution $u(t, x)$ and a terminal condition $u(T, X_T) = \Phi(X_T)$, see e.g., [27, 13, 14]. The relations are as follows:

$$Y_t = u(t, X_t), \quad Z_t = \sigma^\top(t, X_t) \nabla u(t, X_t). \quad (2.12)$$

The first expression in Eq. (2.12) is called nonlinear Feynman–Kac formula.

Similarly, the RBSDE can be formulated in the Markovian framework with lower bound $L_t = g(t, X_t)$, a generator function $f(t, X_t, Y_t, Z_t)$, and a terminal condition $\xi = \Phi(X_T)$ as follows:

$$\begin{aligned} X_t &= x_0 + \int_0^t b(s, X_s) ds + \int_0^t \sigma(s, X_s) dW_s, \quad 0 \leq t \leq T \\ Y_t &= \Phi(X_T) + \int_t^T f(s, X_s, Y_s, Z_s) ds + C_T - C_t - \int_t^T Z_s dW_s \quad 0 \leq t \leq T \\ Y_t &\geq g(t, X_t) \\ \{C_t\} &\text{ a continuous increasing process, } C_0 = 0, \text{ and } \int_0^T (Y_t - L_t) dC_t = 0. \end{aligned} \quad (2.13)$$

The RBSDE in Eq. (2.13) is related to an obstacle problem for a parabolic PDE, see e.g., [11], with a solution $u(t, x)$ of the form:

$$\begin{cases} \min \left[u(t, x) - g(t, x), -\frac{\partial u}{\partial t}(t, x) - \mathcal{L}(u(t, x)) - f(t, x, u(t, x), (\nabla u \sigma)(t, x)) \right] = 0, \\ u(T, x) = \Phi(x), \quad (t, x) \in (0, T) \times \mathbb{R}^d, \end{cases}$$

where $\mathcal{L}$ is given by:

$$\mathcal{L} = \frac{1}{2} \sum_{i,j=1}^d (\sigma \sigma^\top(x, t))_{i,j} \frac{\partial^2}{\partial x_i \partial x_j} + \sum_{i=1}^d b_i(t, x) \frac{\partial}{\partial x_i}.$$

In pricing and hedging options, X represents the logarithm of an underlying asset price $X = \ln(S)$, when S is modeled as in Eq. (1.2). While, Y is a wealth process or value of the option, Z is related to the hedging strategy, and the terminal condition of the BSDE is related to the terminal option payoff.

Chapter 3

Numerical Schemes for BSDEs

The numerical algorithms for BSDEs consist of time discretization of the processes and numerical approximation procedures. In the first section, we present the discrete time approximations of processes Y and Z using the Euler scheme. The discretization of the forward processes is well studied e.g., in [17, 18] while for the backward processes we refer [6, 30]. Further, brief descriptions of numerical approximation methods are presented in the subsequent sections. One is the regression Monte Carlo approach, adapted from the presentation [18, 23], which approximates the conditional expectations in the BSDE solutions. The other is the Deep BSDE method outlined by [10, 21], where we use DNNs for solving BSDEs numerically.

3.1 Time discretization using the Euler scheme

To apply discrete time approximations using the Euler scheme, let us partition the time space $[0, T]$ into finite, uniform, and deterministic time grids, $0 = t_0 < t_1 < \dots < t_N = T$, where $N \geq 1$. We denote the associated discrete time filtration $\{\mathcal{F}_{t_i}\}_{0 \leq i \leq N} := \sigma(X_{t_j}^N, j \leq i)$ and the conditional expectation $\mathbb{E}_{t_i}[\cdot] = \mathbb{E}[\cdot | \mathcal{F}_{t_i}]$. Let also the time step $\Delta_i^N = t_i - t_{i-1}$ and $\Delta W_i^N = W_{t_i} - W_{t_{i-1}}$ for $i = 1, \dots, N$.

The discrete time approximation of the forward process X given in Eq. (2.1) using the Euler scheme given as:

$$\begin{cases} X_{t_0}^N = X_{t_0} \\ X_{t_i}^N = X_{t_{i-1}}^N + b(t_{i-1}, X_{t_{i-1}}^N) \Delta_i^N + \sigma(X_{t_{i-1}}^N) \Delta W_i^N, \quad \text{for } i = 1, \dots, N. \end{cases} \quad (3.1)$$

Then we construct the continuous time approximation of X as follows:

$$X_t^N = X_{t_{i-1}}^N + b(t_{i-1}, X_{t_{i-1}}^N)(t - t_{i-1}) + \sigma(X_{t_{i-1}}^N)(W_t - W_{t_{i-1}}), \quad \text{for } t \in (t_{i-1}, t_i).$$

To develop a discrete time approximation of the backward component, we consider first the BSDE on a one step interval $[t_i, t_{i+1}]$:

$$\begin{aligned} Y_{t_i} &= Y_{t_{i+1}} + \int_{t_i}^{t_{i+1}} f(s, X_s, Y_s, Z_s) ds - \int_{t_i}^{t_{i+1}} Z_s dW_s \\ &\approx Y_{t_{i+1}}^N + \Delta_i^N f(t_i, X_{t_i}^N, Y_{t_i}^N, Z_{t_i}^N) - \Delta W_i^N Z_{t_i}, \end{aligned}$$

where we denoted the Euler discrete time approximation by $Y_{t_i}^N$. Taking conditional expectations of both sides gives:

$$Y_{t_i}^N = \mathbb{E}_{t_i} \left[Y_{t_{i+1}}^N + \Delta_i^N f(t_i, X_{t_i}^N, Y_{t_i}^N, Z_{t_i}^N) \right], \quad (3.2)$$

because Y is adapted. The expression in Eq. (3.2) is an implicit time discrete approximation because $Y_{t_i}^N$ appears on both sides of the equation. The explicit version of the time discrete approximation of Y is given as:

$$Y_{t_i}^N = \mathbb{E}_{t_i} \left[Y_{t_{i+1}}^N + \Delta_i^N f(t_i, X_{t_i}^N, Y_{t_{i+1}}^N, Z_{t_i}^N) \right].$$

However, the rates of convergence of the explicit and the implicit schemes coincide for Lipschitz driver [23]. The implicit scheme requires a Picard iteration procedure to compute $Y_{t_i}^N, i = 0, \dots, N$, see [31], while the explicit does not. Picard iteration is an iterative approximation method for differential equations which results in sequence of approximations by using previous approximations results to approximate the next one.

We get the discrete time approximation for the Z component of the BSDE solution by multiplying both sides of $Y_{t_i}^N$ by ΔW_i^N and take the conditional expectation w.r.t. $\mathcal{F}_{t_i}$:

$$\begin{aligned} Y_{t_i}^N &= Y_{t_{i+1}} + \int_{t_i}^{t_{i+1}} f(s, X_s, Y_s, Z_s) ds - \int_{t_i}^{t_{i+1}} Z_s dW_s \\ \mathbb{E}_{t_i} [\Delta W_i^N Y_{t_i}^N] &= \mathbb{E}_{t_i} [\Delta W_i^N Y_{t_{i+1}}] + \mathbb{E}_{t_i} \left[\Delta W_i^N \int_{t_i}^{t_{i+1}} f(s, X_s, Y_s, Z_s) ds \right] - \mathbb{E}_{t_i} \left[\Delta W_i^N \int_{t_i}^{t_{i+1}} Z_s dW_s \right] \\ 0 &= \mathbb{E}_{t_i} [\Delta W_i^N Y_{t_{i+1}}] - \mathbb{E}_{t_i} \left[\int_{t_i}^{t_{i+1}} dW_s \int_{t_i}^{t_{i+1}} Z_s dW_s \right] \\ 0 &= \mathbb{E}_{t_i} [\Delta W_i^N Y_{t_{i+1}}] - \mathbb{E}_{t_i} \left[\int_{t_i}^{t_{i+1}} Z_s ds \right] \\ Z_{t_i}^N &= \frac{1}{\Delta_i^N} \mathbb{E}_{t_i} [\Delta W_i^N Y_{t_{i+1}}]. \end{aligned}$$

Putting both components and the terminal conditions together gives the following a Dynamic Programming Equation (DPE):

$$\begin{cases} Y_{t_N}^N = \Phi(X_{t_N}^N), & Z_{t_N}^N = 0 \\ Z_{t_i}^N = \frac{1}{\Delta_i^N} \mathbb{E}_{t_i} [\Delta W_i^N Y_{t_{i+1}}], & i = N-1, \dots, 0 \\ Y_{t_i}^N = \mathbb{E}_{t_i} [Y_{t_{i+1}}^N + \Delta_i^N f(t_i, X_{t_i}^N, Y_{t_{i+1}}^N, Z_{t_i}^N)], & i = N-1, \dots, 0. \end{cases} \quad (3.3)$$

Theorem 5 (The time approximation error [30]). *Under the assumptions (B1)-(B3) listed in Subsection 2.3.1, the time approximation error is:*

$$\sup_{0 \leq t \leq T} \mathbb{E}_{t_i} [|Y_t - Y_t^N|^2] + \int_0^T \mathbb{E} [|Z_t - Z_t^N|^2] \leq C |\Delta_i^N|,$$

for some constant C , where $\Delta_i^N = t_i - t_{i-1}$ and N is number of time steps.

Proof. See [30] for the proof and details. $\square$

Using the flow composition property and the Markov property of stochastic differential equations, stated e.g., in [18, Proposition 4.3.2], Eq. (3.3) can be rewritten as follows:

$$\begin{cases} Y_{t_N}^N = \Phi(X_{t_N}^N), & Z_{t_N}^N = 0 \\ Z_{t_i}^N = \frac{1}{\Delta_i^N} \mathbb{E} [\Delta W_i^N Y_{t_{i+1}}^N(X_{t_{i+1}}^N) | X_{t_i}^N], & i = N-1, \dots, 0 \\ Y_{t_i}^N = \mathbb{E} [Y_{t_{i+1}}^N(X_{t_{i+1}}^N) + \Delta_i^N f(t_i, X_{t_i}^N, Y_{t_{i+1}}^N(X_{t_{i+1}}^N), Z_{t_i}^N(X_{t_i}^N)) | X_{t_i}^N], & i = N-1, \dots, 0. \end{cases} \quad (3.4)$$

In the case of RBSDE there is an additional step to compute to check for the optimal solution, and the equation is as follows:

$$\begin{cases} Y_{t_N}^N = \Phi(X_{t_N}^N), & Z_{t_N}^N = 0 \\ Z_{t_i}^N = \frac{1}{\Delta_i^N} \mathbb{E} [\Delta W_i^N Y_{t_{i+1}}^N(X_{t_{i+1}}^N) | X_{t_i}^N], & i = N-1, \dots, 0 \\ \tilde{Y}_{t_i}^N = \mathbb{E} [\tilde{Y}_{t_{i+1}}^N(X_{t_{i+1}}^N) + \Delta_i^N f(t_i, X_{t_i}^N, Y_{t_{i+1}}^N(X_{t_{i+1}}^N), Z_{t_i}^N(X_{t_i}^N)) | X_{t_i}^N], & i = N-1, \dots, 0 \\ Y_{t_i}^N = \max [\tilde{Y}_{t_i}^N, g(X_{t_i}^N)]. \end{cases}$$

In Eq. (3.4) we have an explicit scheme in time, which requires evaluation of conditional expectations each time step that leads to a high order nesting of conditional expectations. Therefore, we need numerical approximation methods to be able to simulate the processes. In the next section, we discuss the approximation methods.

3.2 The least squares regression Monte Carlo method

One of the approximation procedures of conditional expectations, that we use to transform the DPE in (3.4) into an implementable numerical scheme, is the least squares regression Monte Carlo approach. The LSMC method for approximating conditional expectations was introduced by Longstaff and Schwartz [25] in relation to the pricing of American options. This method was developed for BSDEs by [19, 23].

In this approach the conditional expectations in BSDE solutions are approximated as the $\mathbb{L}_2$ -projection on finite-dimensional sub-spaces, which are spanned by predetermined basis functions. As a result the conditional expectations are converted into solving standard least squares problems. The following brief explanation of method is adapted mainly from [18].

Let us denote the components of $Y_{t_i}^N$ in Eq. (3.4) by square integrable random variables R and O :

$$\begin{aligned} R &:= Y_{t_{i+1}}^N + \Delta_i^N f(t_i, X_{t_i}^N, Y_{t_{i+1}}^N, Z_{t_i}^N), & i = 0, \dots, N \\ O &:= X_{t_i}^N, & i = 0, \dots, N. \end{aligned}$$

Then the conditional expectation becomes:

$$\mathbb{E}[R|O] = \mathfrak{M}(O).$$

The function $\mathfrak{M}$ is referred to as the regression/conditional expectation function in [18] and it is the best estimation of the response R in $\mathbb{L}_2$ sense given an observation O .

Taking M finite pair of samples $(O^{(m)}, R^{(m)}), m \in [1, M]$, the regression function is approximated empirically by $\widetilde{\mathfrak{M}}_M$ using the ordinary least-squares on a finite-dimensional linear subspace Ψ generated by J basis functions $\psi(O)$. Define the space Ψ and a square integrable $\psi \in \Psi$:

$$\Psi = \text{Span}(\phi_1, \dots, \phi_J) = \left\{ \psi : \mathbb{R}^d \mapsto \mathbb{R} \text{ s.t } \exists \alpha \in \mathbb{R}^J \text{ and } \psi(O) := \sum_{j=1}^J \alpha_j \phi_j(O) := \alpha \phi(O) \right\}.$$

Then we look for the basis coefficient that minimizes the empirical square error between the sample responses and the basis functions:

$$\alpha^M = \arg \min_{\alpha \in \mathbb{R}^J} \frac{1}{M} \sum_{m=1}^M \left(R^{(m)} - \alpha \phi(O^{(m)}) \right)^2$$

and set the regression function estimator:

$$\widetilde{\mathfrak{M}}_M(O) = \alpha^M \phi(O^{(m)}).$$

The algorithm for simulating the DPE in Eq. (3.4) can be summarized as follows:

- simulate M independent copies of $\Delta W_i \sim N(0, \Delta_i^N), i = 1, \dots, N$,
- simulate M independent identically distributed trajectories of $X_{t_i}^{(N,m)}, m = 1, \dots, M, i = 0, \dots, N$, and N is number of time steps,
- set the terminal condition

$$\widehat{Y}(t_N, X_{t_N}^{(N,m)}) = \Phi(X_{t_N}^{(N,m)}),$$

- choose the basis functions for estimating the $Y_{t_i}^N$ and $Z_{t_i}^N$; denote by $\phi_0(X_{t_i}^{(N,m)})$ and $\phi_1(X_{t_i}^{(N,m)})$, respectively,
- solve the optimal basis coefficients

$$\begin{aligned} \alpha_0^M &= \arg \min_{\alpha \in \mathbb{R}^J} \frac{1}{M} \sum_{m=1}^M \left| \widehat{Y}(t_{i+1}, X_{t_{i+1}}^{(N,m)}) + \Delta_i^N f(t_i, X_{t_i}^{(N,m)}, \widehat{Y}(t_{i+1}, X_{t_{i+1}}^{(N,m)})) - \alpha_0 \phi_0(X_{t_i}^{(N,m)}) \right|^2 \\ \alpha_1^M &= \arg \min_{\alpha \in \mathbb{R}^J} \frac{1}{M} \sum_{m=1}^M \left| \frac{1}{\Delta_i^N} \widehat{Y}(t_{i+1}, X_{t_{i+1}}^{(N,m)}) \Delta W_i^N - \alpha_1 \phi_1(X_{t_i}^{(N,m)}) \right|^2, \end{aligned}$$

- get the optimal α_0^M and α_1^M using one of the techniques used to solve the least-squares problem uniquely, e.g., the singular value decomposition (SVD),
- set the estimates $\widehat{Y}_{t_i}(t_i, X_{t_i}^N) = \alpha_0 \phi_0(X_{t_i}^{(N,m)})$ and $\widehat{Z}_{t_i}(t_i, X_{t_i}^N) = \alpha_1 \phi_1(X_{t_i}^{(N,m)})$,
- iterate from $i = N - 1, \dots, 0$.

The choice of basis functions can vary depending on the problem at hand; global polynomials, hyper-cubes and Voronoi partitions are among the basis functions used in [19]. Hyper-cubes, for example, are defined in [19, 23] as follows.

- First, we choose a domain $D \subset \mathbb{R}^d$ centered on $X_0 = x$, that is, $D = \prod_{i=1}^d [x_i - a, x_i + a]$.
- Then, we partition it into small hyper-cubes of edge δ , such that $D = U_{i_1, \dots, i_d} D_{i_1, \dots, i_d}$ where

$$D_{i_1, \dots, i_d} = [x_1 - a + i_1 \delta, x_1 - a + (i_1 + 1) \delta] \times \dots \times [x_d - a + i_d \delta, x_d - a + (i_d + 1) \delta].$$

- Finally, we define $\phi(X_{t_i}^{(N,m)})$ as the indicator functions associated with the set of hyper-cubes

$$\phi(X_{t_i}^{(N,m)}) = (\mathbf{1}_{D_{i_1, \dots, i_d}})_{i_1, \dots, i_d}.$$

Here the bases are only the indicator functions multiplied by a constant (polynomials of order 0). We can also multiply the indicator functions with polynomials of higher order for a local polynomial bases of higher orders, if necessary.

The global error on the LSMC algorithm depends on the number of time steps N , the number of simulations M and the dimension of the of basis functions; denote the dimension of of basis function for estimating Y_t and Z_t by d_y and d_z , respectively. In [18], the error as a result of approximation of the conditional expectations by LSMC is given as a bias/variance decomposition:

$$\mathbb{E} \left[\left| \widetilde{\mathfrak{M}}_M - \mathfrak{M} \right|_{L_2(\mu^M)}^2 \right] \leq \sigma^2 \frac{J}{M} + \min |\psi - \mathfrak{M}|_{L_2(\mu^M)}^2, \quad (3.5)$$

where $\sigma^2 = \sup_{o \in \mathbb{R}^d} \mathbb{V}ar(R|O = o) < \infty$, μ the marginal distribution of O and $|\cdot|_{L_2(\mu^M)}$ the associated L_2 norm.

E.Gobet [18] states that $\sigma^2 \frac{J}{M}$ corresponds to the statistical error incurred while calculating the coefficients α^M using a finite sample. It converges to 0 as the size M of the sample increases. The second term is due to the approximation error in the space Ψ . This term converges also to 0 as the space of the approximation $J \rightarrow +\infty$.

3.3 Approximation with artificial neural networks

Let us start with an introduction and definitions of some of the terminologies that are frequently used in relation to the deep neural network approximation method.

3.3.1 Artificial neural network introduction

An artificial neuron is a function f_j which takes inputs $I = (I_1, \dots, I_n)$ weighted by connection weights $W = (W_1, \dots, W_n)$, a neuron bias term, and associated to an activation function φ :

$$\text{Output} = f_j(I) = \varphi \left(\sum_j^n (IW + \text{Bias}) \right).$$

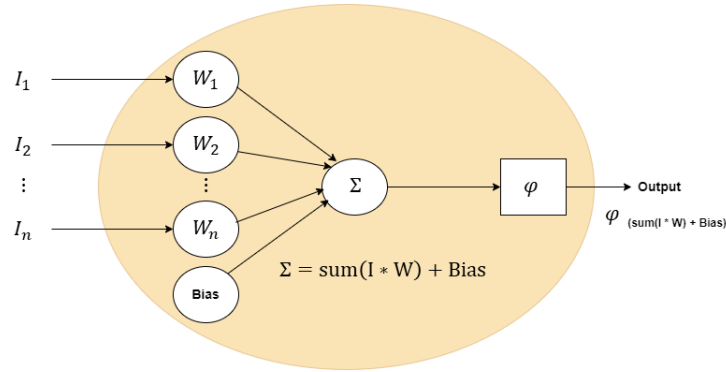


Figure 3.1: An artificial neuron diagram

Figure 3.1 visualises a typical artificial neuron structure. The activation functions are used to approximate nonlinear problems.

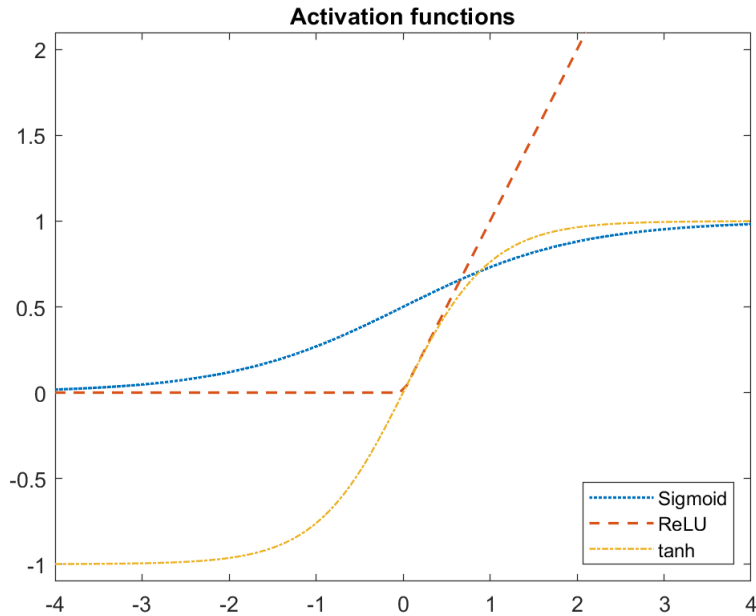


Figure 3.2: Activation functions

Some of mostly used activation functions include:

- the sigmoid function: $\varphi(x) = \frac{1}{1+\exp(-x)}$,
- the hyperbolic tangent function (tanh): $\varphi(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$, and
- the Rectified Linear Unit (ReLU) activation function: $\varphi(x) = \max(x, 0)$.

The shapes of these 3 functions are as shown in Figure 3.2.

All the input neurons form the input layer and networks with more that 2 hidden layers are called deep neural networks. A multilayer neural network is a structure composed of several hidden layers, where the output of a neuron of a layer becomes the input of a neuron of the next layer.

Definition 8 (Fully connected neural network [15]). When all the neurons in a layer are connected to every neuron in the previous layer (i.e., its input neurons), it is called a fully connected layer or a dense layer.

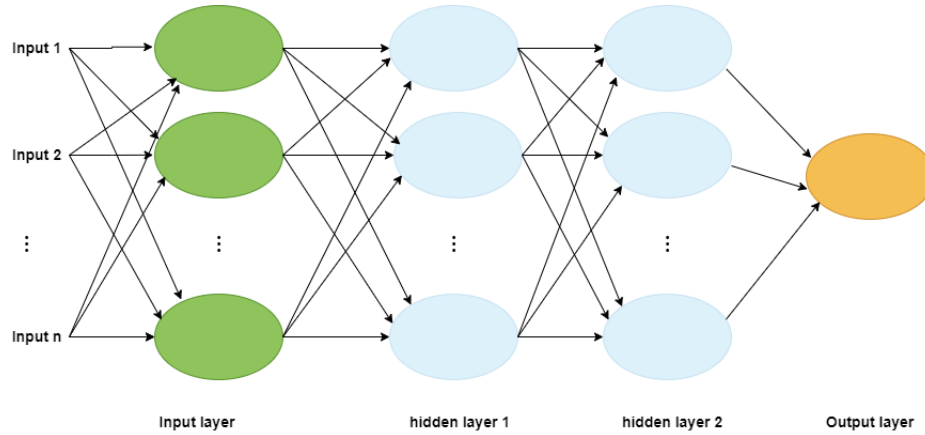


Figure 3.3: Fully connected network with 1 input layer, 2 hidden layers and 1 output layer.

Figure 3.3 shows a densely connected neural network architecture with 1 input layer, 2 hidden layers and 1 output layer. Once the architecture of the network has been chosen, the weights and biases are estimated from a learning samples. This done by minimizing a loss function with a Stochastic Gradient Descent (SGD) algorithm. The optimization is usually done in batches and a batch normalization is applied to speed up the process by keeping the mean and standard deviation of samples in the batch close to 0 and 1, respectively.

3.3.2 The deep BSDE-solver

In this subsection we present an algorithm called a deep BSDE-solver which was proposed by [10, 21]. The authors make an analogy between the BSDE and reinforcement learning to solve the general class of nonlinear parabolic PDEs and BSDEs numerically. The parabolic PDEs

are formulated as BSDEs through nonlinear Feynman–Kac formula. The control process Z in the BSDE serves as a policy function. The pair (Y_0, Z_0) is obtained by training a DNN so as to match the terminal condition.

Using the relation of BSDEs and nonlinear parabolic PDEs we have discussed in Section 2.4, we substitute the solutions in Eq. (2.12) to the BSDE:

$$\begin{aligned} u(T, X_T) - u(t, X_t) = & - \int_t^T f(s, X_s, u(s, X_s), \sigma^\top(s, X_s) \nabla u(s, X_s)) ds \\ & + \int_t^T [\nabla u(s, X_s)]^\top \sigma(s, X_s) dW_s. \end{aligned}$$

Then we write the time discretized solution that corresponds to $Y_{t_i}^N$, in Section 3.1, rearranged to propagate forward in time from t_0 to t_{N-1} :

$$\begin{aligned} u(t_{i+1}, X_{t_{i+1}}) \approx & u(t_i, X_{t_i}^N) - f\left(t_i, X_{t_i}^N, u(t_i, X_{t_i}^N), \sigma^\top(t_i, X_{t_i}^N) \nabla u(t_i, X_{t_i}^N)\right) \Delta t_i \\ & + [\nabla u(t_i, X_{t_i}^N)]^\top \sigma(t_i, X_{t_i}^N) \Delta W_i \quad i = 0, \dots, N-1, \end{aligned} \quad (3.6)$$

considering same time grids as in Section 3.1, $0 = t_0 < t_1 < \dots < t_N = T$.

Since the solution $u(t_i, X_{t_i}^N)$ is not known multilayer feedforward neural networks are employed to approximate the function $\sigma^\top(t, x) \nabla u(t, x)$ at each time step $t = t_i, i = 1, \dots, N-1$:

$$(\sigma^\top \nabla u)(t_i, X_{t_i}^N) \approx (\sigma^\top \nabla u)(t_i, X_{t_i}^N | \theta_i), \quad (3.7)$$

where $\theta_i = \{\theta_1, \dots, \theta_{N-1}\}$ are neural network parameters and we denote $\sigma^\top(t, x) \nabla u(t, x)$ by $(\sigma^\top \nabla u)(t_i, X_{t_i}^N)$ for brevity. Then we compute $u(t_i, X_{t_i}^N)$ recursively using Eq. (3.6) and the deep learning approximations in Eq. (3.7).

The solutions $Y_{t_0}^N \approx u(t_0, X_{t_0}^N)$ and $Z_{t_0}^N \approx (\sigma^\top \nabla u)(t_0, X_{t_0}^N)$ (denoted by θ_{u_0} and $\theta_{\nabla u_0}$, respectively as neural network parameters) are treated as parameters which need to be learned in the course of training. Where as $Z_{t_i}^N, i = 1, \dots, N-1$ are approximated by neural networks which maps $X_{t_i}^N \mapsto (\sigma^\top \nabla u)(t_i, X_{t_i}^N)$. The total set of parameters to be learned and approximated by DNNs are:

$$\theta = \{\theta_{u_0}, \theta_{\nabla u_0}, \theta_1, \dots, \theta_{N-1}\}.$$

The network is trained to minimize the expected mean squared error between the terminal condition and approximated solution of the BSDE, which we call a *loss function*:

$$\ell(\theta) = \mathbb{E} \left[\Phi(X_{t_N}) - \hat{u} \left(\{X_{t_i}^N\}_{0 \leq i \leq N}, \{W_{t_i}\}_{0 \leq i \leq N} \right) \right]^2, \quad (3.8)$$

where we denote the output that approximates $u(t_N, X_{t_N})$ by $\hat{u} \left(\{X_{t_i}^N\}_{0 \leq i \leq N}, \{W_{t_i}\}_{0 \leq i \leq N} \right)$. To find the optimal θ , we use a SGD algorithm.

The steps are summarized as follows.

- Initialize the parameters $u(0, x)$ and $\nabla u(0, x)$ at time $t_0 = 0$ using uniform or normal random numbers.

- Generate $\{\Delta W_i\}_{0 \leq i \leq N} \sim N(0, \Delta t_i)$.
- Simulate M samples of the forward process $\{X_{t_i}^N\}_{0 \leq i \leq N}$ according to Eq. (3.1); both $\{\Delta W_i\}_{0 \leq i \leq N}$ and $\{X_{t_i}^N\}_{0 \leq i \leq N}$ serve as input data.
- Approximate the gradient $\nabla u(t_i, X_{t_i}^N)$ by parameters of the DNN θ_i propagating forward from $i = 1, \dots, N - 1$.
- Stack the approximated results according to Eq. (3.6).
- Employ a SGD algorithm to get the optimal θ that minimizes the loss function in Eq. (3.8).

The implementation of the deep BSDE-solver utilizes $N - 1$ fully-connected feedforward neural networks each consisting of 4 layers. The layers comprise of 1 d -dimensional input layer, $(d + 10)$ -dimensional 2 hidden layers, and 1 d -dimensional output layer. Visualization of the architecture is presented in Figure 3.4, where $h_i^j, \leq j \leq H$ are hidden layers at time steps $\{t_i\}_{1 \leq i \leq N-1}$ and H is number of hidden layers at each steps.

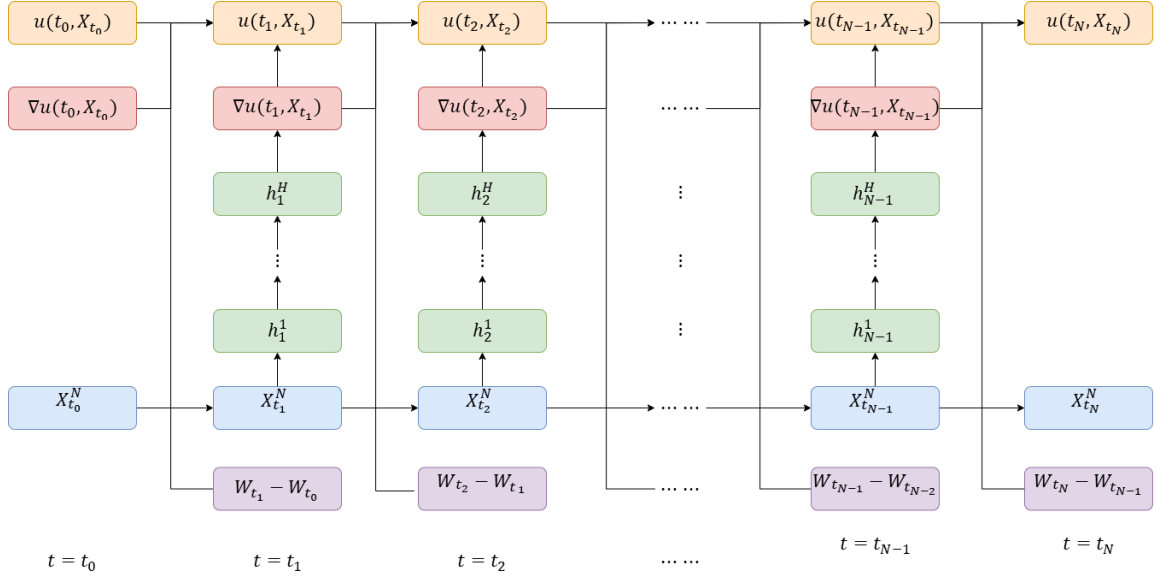


Figure 3.4: Architecture of the deep BSDE-solver

At the bottom part of Figure 3.4, we have the input layer $W_{t_1} - W_{t_0}, \dots, W_{t_N} - W_{t_{N-1}}$ and $X_{t_0}^N, \dots, X_{t_N}^N$ which represent the Wiener increments and simulation of $\{X_{t_i}^N\}_{0 \leq i \leq N}$. The feedforward DNN are represented by $\{X_{t_i}^N \rightarrow h_i^j \rightarrow \nabla u(t_i, X_{t_i}^N)\}_{1 \leq i \leq N-1}$ for $1 \leq j \leq H$. The output layer is represented by $(u(t_i, X_{t_i}^N), \nabla u(t_i, X_{t_i}^N), W_{t_{i+1}} - W_{t_i})$ which is constructed according to Eq. (3.6) and uses the DNN approximation given by Eq. (3.7).

Chapter 4

Implementation of F-BSDEs to Option Pricing and Hedging Problems

Pricing problems of vanilla and call spread options are formulated using the F-BSDE for both European and American options. We consider also higher interest rate for borrowing than the risk-free rate.

4.1 Formulation of the European option pricing and hedging problem as a F-BSDE

Let us consider the financial market defined in Eqs. (1.1) - (1.2), and let $S^i = S$ such that the forward process becomes $X = \ln(S)$:

$$dX_t = \mu_t dt + \sigma_t dW_t.$$

We can construct a portfolio by investing π amount of the wealth on stocks and the remaining on the money market account. Then the wealth process Y of a self-financing portfolio evolves according to:

$$\begin{aligned} Y_t &= Y_0 + \int_0^t \pi_s dX_s + \int_0^t (Y_s - \pi_s) r ds \\ Y_t &= Y_0 + \int_0^t [\pi_s (\mu_s - r_s) + r_s Y_s] ds + \int_0^t \pi_s \sigma_s dW_s. \end{aligned}$$

In option pricing, the terminal payoff function $\Phi(X_T)$ is known instead of the initial value Y_0 . Therefore, we formulate the problem as a BSDE:

$$Y_t = \Phi(X_T) - \int_t^T (Z_s \lambda_s + r_s Y_s) ds - \int_t^T Z_s dW_s, \quad (4.1)$$

where $Z_t := \pi_t \sigma_t$ and $\lambda_t := \sigma_t^{-1}(\mu_t - r_t \mathbf{1})$; compare with the last expression for dV in Definition 1. This is a linear BSDE and has a well known explicit solution, namely the Black–Scholes formula [5].

The implementation of the BSDE algorithm is more appropriate in case of nonlinear problems. Let us consider a nonlinear pricing problem presented, e.g., in [9, 4]. In the problem we have different interest rates where one can invest in the money market account at an interest rate r but can borrow at an interest rate $R > r$.

Since the borrowing rate is higher than the risk-free rate, i.e. $R > r$, a rational investor will not borrow money and invest it in the money market account or bonds at the same time. Therefore the amount borrowed at time t will be restricted to $\min(Y_t - \sum_{i=1}^d \pi_t^i, 0)$. Accordingly, we add the following expression to the generator function in Eq. (4.1):

$$-\max(Y_t - \pi_t, 0)r dt + \min(Y_t - \pi_t, 0)R dt \iff (R - r) \min(Y_t - \pi_t, 0) dt,$$

for the extra cost we incur due to higher borrowing rate than the risk-free rate. The wealth process and the corresponding hedging portfolio becomes:

$$Y_t = \Phi(X_T) - \int_t^T [rY_s + \lambda Z_s - (R - r) \min\{Y_s - \sigma^{-1}Z_s, 0\}] ds - \int_t^T Z_s dW_s. \quad (4.2)$$

The payoff function for the European vanilla call option is

$$\Phi(X_T) = \left\{ \max_{i=1, \dots, d} (e^{X_T}, 0) - K, 0 \right\}^+,$$

where d is the number (dimension) of the risky assets involved. In this case, we might not need complicated numerical algorithms since the hedging problem can be solved in a standard Black–Scholes setting with a bank account given by e^{Rt} , see [2]. If we consider a call spread option, however, we can notice the impact of nonlinearity of the generator function clearly as it will be justified by the numerical experiment. Therefore employing numerical algorithms to solve the F-BSDE becomes necessary in this case.

Given strike prices K_1 and K_2 , the payoff for a call spread option is given by:

$$\Phi(X_T) = \left\{ \max_{i=1, \dots, d} (e^{X_T}, 0) - K_1 \right\}^+ - 2 \left\{ \max_{i=1, \dots, d} (e^{X_T}, 0) - K_2 \right\}^+. \quad (4.3)$$

4.2 Formulation of the American option pricing and hedging problem as a F-BSDE

We formulate the American option pricing problem by considering the same financial market and portfolio as in Section 4.1. As we have discussed in Section 1.2, the holder of the American call option has the right to buy the underlying asset at a given strike price K at any time until maturity. Therefore, we need to introduce a process C to the value of a replicating portfolio for the American option $Y_t, t \in [0, T]$ which represents the premium for the right to exercise the option at a favorable time. The process Y must also satisfy a constraint, e.g., $Y_t \geq \left\{ \max_{i=1, \dots, d} (e^{X_t}, 0) - K, 0 \right\}^+$ for the vanilla call option. Consequently, the problem of finding a replicating portfolio and a replicating strategy for the American option $(Y_t, \pi_t \sigma_t)$

corresponds to the solution of Markovian RBSDE, as studied in [12]. The obstacle in the Markovian RBSDE Eq. (2.13) corresponds to the constraint to the value of the American option, e.g., $g(t, X_t) = \{\max_{i=1, \dots, d} (e^{X_t}, 0) - K, 0\}^+$ for vanilla call option. The American vanilla option can be formulated in the Markovian RBSDE setting as follows:

$$\begin{cases} Y_t = \Phi(X_T) - \int_t^T (Z_s \lambda_s + r_t Y_s) ds + C_T - C_t - \int_t^T Z_s dW_s & 0 \leq t \leq T \\ Y_t \geq g(t, X_t) \\ \{C_t\} \text{ a continuous increasing process, } C_0 = 0, \text{ and } \int_0^T (Y_t - g(t, X_t)) dC_t = 0. \end{cases} \quad (4.4)$$

In the case of the call spread option the terminal payoff function can be replaced by Eq. (4.3), while the generator function, and the obstacle in Eq. (4.4) can be replaced by the following:

$$\begin{aligned} f(t, X_t, Y_t, Z_t) &= - \left(rY_t + \lambda Z_t - (R - r) \min \{Y_t - \sigma^{-1} Z_t, 0\} \right) \\ g(t, X_t) &= \left\{ \max_{i=1, \dots, d} (e^{X_t}, 0) - K_1 \right\}^+ - 2 \left\{ \max_{i=1, \dots, d} (e^{X_t}, 0) - K_2 \right\}^+. \end{aligned}$$

The imposition of the constraint $\int_0^T (Y_t - g(t, X_t)) dC_t = 0$ in the RBSDE can be related to the fact that the American option has time value which decreases as time evolves and the right to exercise the option should only be executed if the price of the option is equal to $g(t, X_t)$. We do not exercise at time t if $Y_t > g(t, X_t)$ which can be interpreted as there is no time value at t and consequently, $dC_t = 0$ when $Y_t > g(t, X_t)$, see [31].

If we choose to exercise the option at a stopping time $\tau \geq t$ the payoff at τ is:

$$\tilde{L}_\tau = g_\tau \mathbb{1}_{\{\tau < T\}} + \Phi(X_T) \mathbb{1}_{\{\tau = T\}}.$$

In this case, solutions of BSDE (Y, Z) with terminal condition $\tilde{L}_\tau$ and maturity $T = \tau$ will be solutions to the American option. Since we can only make decisions based on the information available, the price of the American contingent claim is given as an optimal stopping time problem:

$$Y_t = \text{ess sup}_{\tau \in \mathcal{T}_{[t, T]}} \mathbb{E} \left[\int_t^\tau f(s, Y_s, Z_s) ds + g_\tau \mathbb{1}_{\{\tau < T\}} + \Phi(X_T) \mathbb{1}_{\{\tau = T\}} \mid \mathcal{F}_t \right],$$

where $\mathcal{T} := \{\tau \in [t, T]\}$ is set of stopping times. For the Lipschitz and convex generator function f , the existence and uniqueness of this solution is proven, e.g., in [11].

4.3 Numerical experiment

For the numerical experiments, we consider a call spread option with parameters $S_0 = 100$, $\mu = 0.05$, $\sigma = 0.2$, $T = 0.25$, $r = 0.01$, $R = 0.06$, $K_1 = 95$, $K_2 = 105$ and the dimension $d = 1$. The values are chosen to be same as in examples used in [19, 23] for the sake of comparison. The experiments are carried out on a Windows 10 machine with a 3 GHz Intel Core i5 processor and 16 GB memory. In the next subsection we briefly describe the procedures that we follow for this specific implementation, we present and discuss the results.

4.3.1 Presentation and discussion of results

The LSMC method

As basis functions, we choose global polynomial and local polynomials of order 0 in hypercubes. The hypercubes are defined on the interval $[40, 180]$ one with edge $\delta = 5$ and time steps $N = 5$ and a second hypercube with edge $\delta = 1$ and time steps $N = 10$. The global polynomial bases used are polynomials of orders $0, \dots, 3$ and the first $0, \dots, 3$ series of Laguerre polynomials. The Laguerre polynomials have the form:

$$\begin{aligned} L_n(x) &= \sum_{k=0}^n \binom{n}{k} \frac{(-1)^k}{k!} x^k, \\ L_0(x) &= 1, \\ L_1(x) &= 1 - x, \\ L_2(x) &= \frac{1}{2} (x^2 - 4x - 2), \\ L_3(x) &= \frac{1}{6} (-x^3 + 9x^2 - 18x + 6), \\ &\vdots \end{aligned}$$

All basis functions are executed for different number of iterations M as shown in the tables below. Each bases at the given M run 50 times to get the mean, the standard deviation and the 95% confidence interval of 50 the samples.

In Section 4.1, we have discussed that when considering a vanilla option the result we get using the BSDE converges to the value we obtain by the Black–Scholes equation using rate R . To assess this, we take a strike price $K = 100$, while all other parameter values are as described above. The Black–Scholes equation, using R for the interest rate, gives a vanilla European call price equal to 4.7469. Table 4.1 shows the mean, standard deviations and the 95% confidence intervals of 50 European vanilla call option prices obtained using local polynomials (polynomials of order 0 multiplied by the indicator function of the hypercubes) at different number of simulations M , time steps N and hypercube edges δ .

Table 4.1: European vanilla call option (price, standard deviation)

M	$N = 5, \delta = 5$		$N = 10, \delta = 1$	
	(mean, std)	95% conf. interval	(mean, std)	95% conf. interval
128	(4.834381, 0.631000)	[4.725066, 4.943696]	(4.791548, 0.636637)	[4.681256, 4.901839]
512	(4.762802, 0.295411)	[4.737213, 4.788391]	(4.778966, 0.275860)	[4.755071, 4.802861]
2048	(4.734678, 0.145298)	[4.728385, 4.740971]	(4.798227, 0.130969)	[4.792554, 4.803899]
8192	(4.751151, 0.081170)	[4.749393, 4.752908]	(4.760030, 0.059688)	[4.758737, 4.761322]
32768	(4.743019, 0.030193)	[4.742692, 4.743346]	(4.750726, 0.032820)	[4.750370, 4.751081]

From Table 4.1, we can notice that almost all 95% confidence intervals capture the value we obtained using the Black–Scholes equation at interest rate R .

Next we test linear combination of call prices using the Black–Scholes formula by alternating the parameters K_1, K_2, R, r to compute the European call spread option. Our reference price for this call spread option is 2.95 which is shown in [19]. From Table 4.2, we can see that the first three rows under estimate the price while the last row is an over estimate. This makes apparent that the generator function being nonlinear has an impact on this computation.

Table 4.2: European call spread using the Black–Scholes equation

$BS(K_1, R) - 2BS(K_2, R)$	2.7503
$BS(K_1, r) - 2BS(K_2, r)$	2.7649
$BS(K_1, r) - 2BS(K_2, R)$	1.9159
$BS(K_1, R) - 2BS(K_2, r)$	3.5993

None of the values in Table 4.2 appear in the 95% confidence intervals of same problem, obtained using the global and local polynomials, shown in Tables A.1 and A.3.

Let d_y and d_z denote the number of polynomials used in the global polynomial basis functions for approximating Y_t and Z_t , respectively. In Table 4.3, we tabulate the mean of 50 European call spread option prices and hedging strategies, computed using global polynomial bases with up to 3 orders of polynomials and the first 3 Laguerre polynomials, at different time steps and number of iterations. Moreover, the standard deviations and the 95% confidence intervals of the 50 sample prices are shown in Tables A.1 and A.2.

Table 4.3: European call spread using global polynomial basis functions

M	Laguerre polynomials		Polynomials of order 0, ..., 3	
	$N = 20$	$N = 50$	$N = 20$	$N = 50$
	$d_y = 2, d_z = 1$	$d_y = 4, d_z = 2$	$d_y = 4, d_z = 2$	$d_y = 5, d_z = 5$
	mean ($\hat{Y}_0, \pi$)	mean ($\hat{Y}_0, \pi$)	mean ($\hat{Y}_0, \pi$)	mean ($\hat{Y}_0, \pi$)
128	(2.8977, -2.7080)	(2.8670, -3.3865)	(2.8977, -2.7080)	(2.8670, -3.3865)
512	(2.8347, 0.1769)	(2.7969, 0.2116)	(2.8347, 0.1769)	(2.7969, 0.2116)
2048	(2.8088, -0.4672)	(2.7853, -0.7272)	(2.8088, -0.4672)	(2.7853, -0.7272)
8192	(2.8193, -0.6018)	(2.7682, -0.6920)	(2.8193, -0.6018)	(2.7682, -0.6920)
32768	(2.8081, -0.4646)	(2.7683, -0.9096)	(2.8081, -0.4646)	(2.7683, -0.9096)

If we consider the minimum and the maximum values we get using the linear combinations of the Black–Scholes equation as a lower and upper bounds, respectively, all estimations in Table 4.3 are within the interval [1.9159, 3.5993]. We do not notice, however, any kind of pattern in the price as we change any of the parameters. The standard deviation, on the other hand, becomes obviously smaller as the number of iteration increases. Surprisingly, although we used two different types of polynomials, we got identical values at same number of iterations and number polynomials involved in each bases. Consequently, we can conclude that what matters is the order of the polynomials. We could not implement polynomials of order more than 5 on the computer details specified at the beginning of this section. In [19], however, the authors could get 2.95 when $d_y = 9, d_z = 9, N = 50$. From this, we notice that it is possible to

achieve the target value with global polynomials using as high as $d_y, d_z \geq 9$ order polynomials, which results in a very high complexity.

Table 4.4 shows the mean prices and hedging strategies of 50 samples found using local polynomial (polynomials of order 0 multiplied by the indicator function of the hypercube) on hypercubes $[40, 180]$ with edge $\delta = 5$ and $N = 5$ in the first column and $\delta = 1$ and $N = 10$ in the second column, respectively. The prices seem to have better accuracy compared to the global polynomials; especially the second column affirms the claim. The standard deviations and the 95% confidence intervals are also presented in Table A.3.

Table 4.4: European call spread using local polynomials of order 0

	$N = 5, \delta = 5$	$N = 10, \delta = 1$
M	mean $(\hat{Y}_0, \pi)$	mean $(\hat{Y}_0, \pi)$
128	(2.931727, -0.123849)	(3.056163, -0.165910)
512	(2.923739, -0.183301)	(2.970024, -0.134521)
2048	(2.919599, -0.112260)	(2.943109, -0.102257)
8192	(2.911789, -0.096771)	(2.938153, -0.026258)
32768	(2.915952, -0.123509)	(2.943210, -0.013705)

When we increase the time step N and the number of hypercubes (decrease δ), the approximation gets clearly closer to the solution 2.95; thanks to a smaller discretization error and a smaller approximation error. We can note also that the bases on hypercubes have considerably better accuracy than the global polynomial bases. Another point worth to note is that the bases on hypercubes do not require higher orders of polynomials as the global polynomials do. In this experiment, the indicator functions of the hyper cubes alone have shown sufficiently good approximations.

As final part of our experiment on the least square method on the European option, we show the convergence/divergence of estimations of Y_0 using the bases on hypercube to the reference value 2.95 as we modify the parameters N , M and the hypercube edge δ . On the fixed domain $[40, 180]$, we let the parameters to vary for $j = 2, \dots, 14$ iterations as follows:

- $N = 2 \left(\sqrt{2} \right)^{(j/2)-1},$
- $\delta = \frac{50}{\left(\sqrt{2} \right)^{((j/2)-1)/2}},$
- $M = 2 \left(\sqrt{2} \right)^{\alpha_M((j/2)-1)}.$

We conduct the test for values of $\alpha_M = 1, \dots, 5$. Figure 4.1 depicts the number of iteration j verses the price when $\alpha_M = 1, \dots, 5$. When $\alpha_M = 1$, Y_0 diverges while it converges slowly to the reference price 2.95 when $\alpha_M = 2$. This can be seen more clearly in Figure 4.2 where we take more iterations, i.e. $j = 2, \dots, 24$. The convergence is faster for $\alpha_M = 3, 4, 5$ for this experiment.

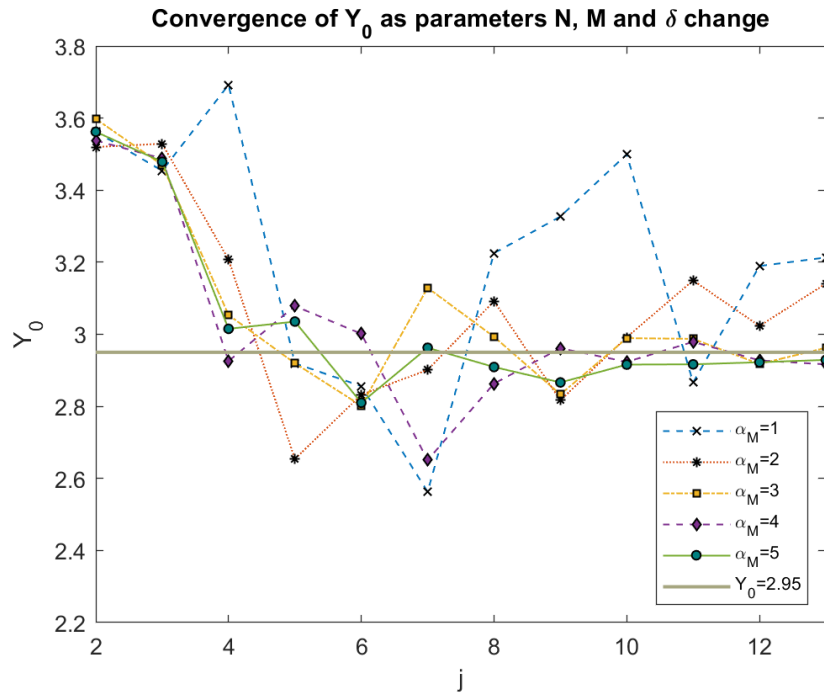


Figure 4.1: Convergence of Y_0 to the expected value as we change the number of iteration M , time steps N , and the hypercube edges δ for $\alpha_M = 1, \dots, 5$.

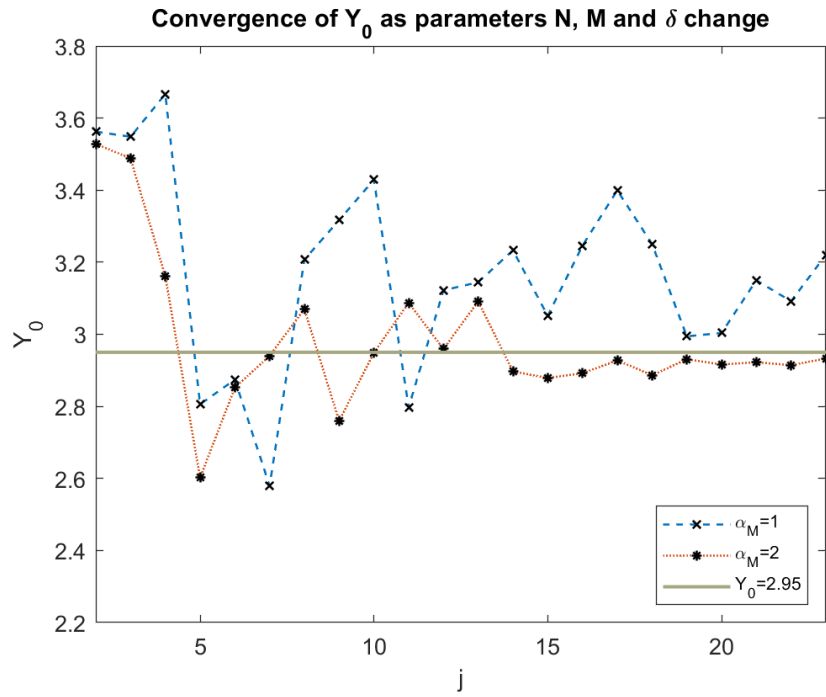


Figure 4.2: Convergence of Y_0 to the expected value as we change the number of iteration M , time steps N , and the hypercube edges δ for $\alpha_M = 1, 2$.

Next, we implement the global polynomial basis functions for approximation of the American call spread option. Table 4.5 shows the mean of 50 American call spread option prices and hedging strategies, computed using global polynomial bases with up to 3 orders of polynomials and the first 3 Laguerre polynomials, at different time steps and number of iterations. Moreover, the standard deviations and the 95% confidence intervals of the 50 sample prices are shown in Tables A.4 and A.5.

Table 4.5: American call spread using global polynomial basis functions

M	Laguerre polynomials		Polynomials of order 0, ..., 3	
	$N = 20$	$N = 50$	$N = 20$	$N = 50$
	$d_y = 2, d_z = 1$	$d_y = 4, d_z = 2$	$d_y = 4, d_z = 2$	$d_y = 5, d_z = 5$
	mean ($\hat{Y}_0, \pi$)	mean ($\hat{Y}_0, \pi$)	mean ($\hat{Y}_0, \pi$)	mean ($\hat{Y}_0, \pi$)
128	(5.1952, 11.0547)	(5.0404, 11.3034)	(5.1952, 11.0547)	(5.0404, 11.3034)
512	(5.1814, 17.5771)	(5.0330, 21.2665)	(5.1814, 17.5771)	(5.0330, 21.2665)
2048	(5.1842, 15.3909)	(5.0342, 19.3083)	(5.1842, 15.3909)	(5.0342, 19.3083)
8192	(5.1778, 15.8313)	(5.0362, 19.9776)	(5.1778, 15.8313)	(5.0362, 19.9776)
32768	(5.1796, 16.1485)	(5.0351, 19.0623)	(5.1796, 16.1485)	(5.0351, 19.0623)

The DNN method

The implementation of the deep BSDE-solver is built using TensorFlow in python. We keep most settings of the solver as default as have been set by the authors in [10] with some modification to the current problem. ReLU applied as activation function for the hidden variables. To optimize the DNN parameters Adam optimizer is employed. Further, the parameters θ_{u_0} and $\theta_{\nabla u_0}$ are initialized using a uniform distribution without any pre-training, as shown in the listing below.

```

self.y_init = tf.Variable(np.random.uniform(low=self.yInitRange[0],
                                           high=self.yInitRange[1],
                                           size=[1]))

self.z_init = tf.Variable(np.random.uniform(low=-.1, high=.1,
                                           size=[1, self.dim]))

```

where `yInitRange= [1, 2]` stands for the initial guess interval of Y_0 . Batch normalization is applied with batch size 64 and the learning rate is set to 0.005 with boundary equal to 2000.

We run the solver [20] for $M = 1000, 2000, \dots, 6000$ number of iterations, each simulation 5 times. Then we tabulate the mean and standard deviation of the target value, the mean and standard deviation of the relative errors and the mean time elapses (in seconds) of the 5 runs in Table 4.6.

Table 4.6: Mean and standard deviations of the target value and relative errors at different number of iterations using the deep BSDE-solver

M	Mean $\hat{u}$	std of $\hat{u}$	mean r.error	std of r.error	mean time elapse in sec.
1000	2.828880	0.024164	1.828880	0.024164	14.00
2000	2.900240	0.013069	1.900240	0.013069	20.40
3000	2.930260	0.016906	0.587155	1.304061	31.4
4000	2.941568	0.028077	1.941568	0.028077	39.80
5000	2.943988	0.011109	1.943988	0.011109	52.60
6000	2.949372	0.020901284	1.949372	0.020901284	81.2

For one dimensional problems, the time elapse is not better than the LSMC method. For example, at $M = 6000$ the time elapsed using the deep BSDE-solver is 81.2 seconds. Whereas, using the hypercube with $N = 5$ and $\delta = 5$ the execution time is 4.168 seconds only while the hypercube with $N = 10$ and $\delta = 1$ takes 52.901 seconds. Significance of the DNN is in higher dimensions. In our experiment of higher dimensions with the deep BSDE-solver, the time elapsed when $d = 10$ is 59 seconds and when $d = 100$ is 105 seconds both at $M = 6000$. All time measurements are mean of 5 runs.

Chapter 5

Conclusion and Future Works

In this project we investigated the Markovian BSDE formulation of pricing and hedging problems of the European and American options. We implemented the approximation methods LSMC and DNN for the European option.

The results from LSMC experiments on the European option suggests that the global polynomial basis functions require higher orders of polynomials, high complexity as a result, while using the local polynomials we can achieve the target value at zero order polynomial. The approximation can, however, take longer as the time steps increase and the hypercube edge decrease. The LSMC method has clear procedures for analysis of error; [19], [23] and [18] are among a number of literature that extensively study the error analysis related to the LSMC method. Although the method is relatively handy to implement in low dimensions, we are able to compute only to certain number of dimensions; up to $d = 3$ e.g., in [19].

Using deep BSDE-solver, on the other hand, we can compute up to $d = 100$ problems within a reasonable time. For one dimensional problems, however, the LSMC method appears to be more efficient. The drawback with the DNN approximation method of BSDEs in general is that it lacks error analysis that has been established for traditional numerical methods. One can find only few recent publications, e.g., [16], in relation to the error analysis to the DNN approximation of BSDEs. A limitation specific to the deep BSDE-solver developed by [10, 21] is that it gives only the time $t = 0$ output. We might be interested to get the prices at some other time points. There are algorithms (upon a single round of training) results in a solution function that can be evaluated anywhere in the space-time domain, e.g., [28]. Moreover, we can use the deep BSDE-solver for the European type option only. For options with stopping time, namely the American and Bermudan type options, we need to develop a backward propagating network. Interested reader might see [16, 22] and [24].

As contribution, in this project we put together the financial theory, studies related to BSDEs and their different methods of approximation related to financial applications. We hope that this will significantly reduce the time and effort for those who start researching in this area.

Bibliography

- [1] V. Bally and G. Pages. Error analysis of the optimal quantization algorithm for obstacle problems. *Stochastic processes and their applications*, 106(1):1–40, 2003.
- [2] C. Bender and J. Steiner. Least-squares Monte Carlo for backward SDEs. In *Numerical methods in finance*, pages 257–289. Springer, 2012.
- [3] F. Benth, K. Karlsen, and K. Reikvam. A semilinear Black and Scholes partial differential equation for valuing American options. *Finance and Stochastics*, 7:277–298, 07 2003. doi: 10.1007/s007800200091.
- [4] Y. Bergman. Option pricing with different interest rates for borrowing and for lending. *Review of Financial Studies*, 8:475–500, 04 1995.
- [5] F. Black and M. Scholes. The pricing of options and corporate liabilities. *Journal of political economy*, 81(3):637–654, 1973.
- [6] B. Bouchard and N. Touzi. Discrete-time approximation and Monte Carlo simulation of backward stochastic differential equations. *Stochastic Processes and their Applications*, 111(2):175–206, 2004. ISSN 0304-4149. doi: <https://doi.org/10.1016/j.spa.2004.01.001>. URL <https://www.sciencedirect.com/science/article/pii/S0304414904000031>.
- [7] B. Bouchard, K. W. Chau, A. Manai, and A. Sid-Ali. Monte Carlo methods for the pricing of American options: A semilinear BSDE point of view. *ESAIM: Proceedings and Surveys*, 65:294–308x, 2019.
- [8] U. Cherubini, E. Luciano, and W. Vecchiato. *Copula methods in finance*. Wiley Finance Series. John Wiley & Sons, Ltd., Chichester, 2004. ISBN 0-470-86344-7. doi: 10.1002/9781118673331. URL <https://doi.org/10.1002/9781118673331>.
- [9] J. Cvitanic and I. Karatzas. Hedging contingent claims with constrained portfolios. *Ann. Appl. Probab.*, 3, 08 1993. doi: 10.1214/aoap/1177005357.
- [10] W. Ee, J. Han, and A. Jentzen. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *To appear in Communications in Mathematics and Statistics*, 5, 06 2017. doi: 10.1007/s40304-017-0117-6.

- [11] N. El Karoui, C. Kapoudjian, E. Pardoux, S. Peng, and M. C. Quenez. Reflected solutions of backward SDE's, and related obstacle problems for PDE's. *The Annals of Probability*, 25(2):702–737, 1997. ISSN 00911798. URL <http://www.jstor.org/stable/2959608>.
- [12] N. El Karoui, E. Pardoux, and M. Quenez. Reflected backward SDEs and American options. *Numerical methods in finance*, 13:215, 1997.
- [13] N. El Karoui, S. Peng, and M. C. Quenez. Backward stochastic differential equations in finance. *Mathematical finance*, 7(1):1–71, 1997.
- [14] N. El Karoui, S. Hamadène, and A. Matoussi. Backward stochastic differential equations and applications. *Indifference pricing: theory and applications*, pages 267–320, 2008.
- [15] A. Gereon. Hands-on machine learning with Scikit-Learn and TensorFlow. *O'Reilly Media Inc., USA*, 2018.
- [16] M. Germain, H. Pham, and X. Warin. Approximation error analysis of some deep backward schemes for nonlinear PDEs. *SIAM Journal on Scientific Computing*, 44: A28–A56, 01 2022. doi: 10.1137/20M1355355.
- [17] P. Glasserman. *Monte Carlo Methods in Financial Engineering*, volume 53. Springer Science & Business Media, 2013. ISBN 978-0-387-00451-8.
- [18] E. Gobet. *Monte Carlo Methods and Stochastic Processes: From Linear to Non-Linear*. Chapman and Hall/CRC, 09 2016. ISBN 9781315368757. doi: 10.1201/9781315368757.
- [19] E. Gobet, J.-P. Lemor, and X. Warin. A regression-based Monte Carlo method to solve backward stochastic differential equations. *The Annals of Applied Probability*, 15(3): 2172–2202, 2005.
- [20] J. Han. Deep BSDE Solver in TensorFlow (2.0). <https://github.com/frankhan91/DeepBSDE>, Accessed May 24, 2022.
- [21] J. Han, A. Jentzen, and W. E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018. doi: 10.1073/pnas.1718942115. URL <https://www.pnas.org/doi/abs/10.1073/pnas.1718942115>.
- [22] C. Huré, H. Pham, and X. Warin. Deep backward schemes for high-dimensional nonlinear PDEs, 2019. URL <https://arxiv.org/abs/1902.01599>.
- [23] J.-P. Lemor, E. Gobet, and X. Warin. Rate of convergence of an empirical regression method for solving generalized backward stochastic differential equations. *Bernoulli*, 12 (5):889–916, 2006.
- [24] J. Liang, Z. Xu, and P. Li. Deep learning-based least squares forward-backward stochastic differential equation solver for high-dimensional derivative pricing. *Quantitative Finance*, 21(8):1309–1323, 2021.

- [25] F. A. Longstaff and E. S. Schwartz. Valuing American options by simulation: A simple least-squares approach. *The review of financial studies*, 14(1):113–147, 2001.
- [26] E. Pardoux and S. Peng. Adapted solution of a backward stochastic differential equation. *Systems & Control Letters*, 14(1):55–61, 1990.
- [27] E. Pardoux and S. Tang. Forward-backward stochastic differential equations and quasi-linear parabolic PDEs. *Probability Theory and Related Fields*, 114(2):123–150, 1999.
- [28] M. Raissi. Forward-backward stochastic neural networks: Deep learning of high-dimensional partial differential equations. *arXiv preprint arXiv:1804.07010*, 2018.
- [29] M. Schroder and C. Skiadas. Optimality and state pricing in constrained financial markets with recursive utility under continuous and discontinuous information. *Mathematical Finance: An International Journal of Mathematics, Statistics and Financial Economics*, 18(2):199–238, 2008.
- [30] J. Zhang. A numerical scheme for BSDEs. *The annals of applied probability*, 14(1): 459–488, 02 2004. doi: 10.1214/aoap/1075828058.
- [31] J. Zhang. *Backward Stochastic Differential Equations*. Springer, New York, NY, 2017. ISBN 978-1-4939-7256-2. URL <https://doi.org/10.1007/978-1-4939-7256-2>.

Appendix A

Matlab Code for the LSMC Method using Global Polynomial Bases

European call spread option with different interest rates for borrowing and lending using Laguerre polynomials as bases

```
% European Call with regression (Laguerre polynomial bases, N=50, dy=4, dz=2)
clc;clear;
rng('default');

%Data
r=0.01;
R=0.06;
mu=0.05;
sigma=0.2;
lambda=(mu-r)/sigma;

%Time grid
T=0.25;
N=50;
t=(T/N):(T/N):T;

% Simulation of M trajectories
M = 8192; % 128, 512, 2048, 8192, 32768

Y0=zeros(1,50); % iterate to get 50 sample Y0s
for k=1:50

    % M independent realizations of B.M
    Wj=zeros(M,N+1);
    delta_Wj=zeros(M,N+1);
    for j=1:M
        delta_Wj(j,:)=[0 sqrt(t-(0:T/N:(T-T/N))).*randn(1,N)];
        Wj(j,:)=cumsum(delta_Wj(j,:));
    end
end
```

```

% M independent realizations of the underlying asset price
S0=100;
Sj=zeros(M,N+1);
for j=1:M
    Sj(j,:)=S0*exp((mu-0.5*sigma^2)*[0 t]+sigma*Wj(j,:));
end

% Strike
K1=95;
K2=105;

%% Basis functions: 4 first Laguerre polynomials
Ly=4;% Dimension of the basis
Lz=2;
L0=1;
L1=@(x) (1-x);
L2=@(x) (x^2-4*x+2)/2;
L3=@(x) (-x^3+9*x^2-18*x+6)/6;
% L4=@(x) (x.^4-16*x.^3+72*x.^2-96*x+24)/24;
pol=@(x) [L0 L1(x) L2(x) L3(x)];% L4(x)];%
polz=@(x) [L0 L1(x)];

% Regression matrix
Mk=zeros(M,Ly);
Mkz=zeros(M,Lz);

% Initialisation of pi (portfolio)
pi=zeros(M,N+1);

% Initialisation of E (Expectation of Y given the filtration)
E=zeros(M,N+1);

%% Initialisation of Y
Yj=zeros(M,N+1);
% Y at the terminal time
Yj(:,N+1)=max(Sj(:,N+1)-K1,0)- 2*max(Sj(:,N+1)-K2,0);

%%
i=N;
while i>1
    % Regression matrix
    for m=1:M
        Mk(m,:)=pol(Sj(m,i+1));
        Mkz(m,:)=polz(Sj(m,i+1));
    end

    % Least squares for pi
    Lzz=chol(Mkz'*Mkz)'; % Cholesky decomposition
    zz=Lzz\Mkz' * ( Yj(:,i+1).*delta_Wj(:,i+1));
    alphaz=Lzz'\zz; % Regression coefficients

```

```

% calculate estimator of pi
pi(:,i)=Mkz*alphaz/(T/N);

% Least squares for Y
Lyy=chol(Mk'*Mk)'; % Cholesky decomposition
zy=Lyy\Mk'*(Yj(:,i+1));
alpha=Lyy\zy; % Regression coefficients

% Calculate estimator for E(i+1)=E[Yi+1 | Fi]
E(:,i)=Mk*alpha;

%% Picard iterations to evaluate the estimator of Y
J=3;
for j=1:J
    Yj(:,i)=E(:,i)+(-r*Yj(:,i)-pi(:,i)*lambda ...
        +(R-r)*min(Yj(:,i)-pi(:,i),0))*(T/N);
end
i=i-1;
end

pi=sum(Yj(:,2).*delta_Wj(:,2))/(M*(T/N));

% Picard iterations to evaluate the estimator of Y
Y=0;
J=10;
for j=1:J
    Y=sum(Yj(:,2))/M+(-r*Y-pi(:,i)*lambda+(R-r)*min(Y-pi,0))*(T/N);
end

Y0(k)=Y;
mean_Y0=mean(Y0);
std_Y0=std(Y0);

%% 95% confidence interval
upper = mean_Y0 + norminv(0.975) * std_Y0/sqrt(M);
lower = mean_Y0 + norminv(0.025) * std_Y0/sqrt(M);
CI=[lower,upper];
end

for m=1:M
    Yj(m,1)=mean_Y0;
end

[mean_Y0 pi std_Y0 CI]

```

American call spread option with different interest rates for borrowing and lending using Laguerre polynomials as bases

```
% American Call with regression (Laguerre polynomial bases, N=50, dy=4, dz=2)
clc;clear;%clf;
rng('default');

%Data
r=0.01;
R=0.06;
mu=0.05;
sigma=0.2;
lambda=(mu-r)/sigma;

%Time grid
T=0.25;
N=50;
t=(T/N):(T/N):T;

% Simulation of M trajectories
M=8192; % 128, 512, 2048, 8192, 32768

Y0=zeros(1,50);
for k=1:50
    % M independent realizations of B.M
    Wj=zeros(M,N+1);
    delta_Wj=zeros(M,N+1);
    for j=1:M
        delta_Wj(j,:)=[0 sqrt(t-(0:T/N:(T-T/N))).*randn(1,N)];
        Wj(j,:)=cumsum(delta_Wj(j,:));
    end

    % M independent realizations of the underlying asset price
    S0=100;
    Sj=zeros(M,N+1);
    for j=1:M
        Sj(j,:)=S0*exp((mu-0.5*sigma^2)*[0 t]+sigma*Wj(j,:));
    end

    % Strike
    K1=95;
    K2=105;

    %% Basis functions: 4 first Laguerre polynomials
    Ly=4;% Dimension of the basis
    Lz=2;
    L0=1;
    L1=@(x) (1-x);
    L2=@(x) (x^2-4*x+2)/2;
    L3=@(x) (-x^3+9*x^2-18*x+6)/6;
```

```

%      L4=@(x) (x.^4-16*x.^3+72*x.^2-96*x+24)/24;
pol=@(x) [L0 L1(x) L2(x) L3(x)]; % L4(x)]; %
polz=@(x) [L0 L1(x)];

% Regression matrix
Mk=zeros(M,Ly);
Mkz=zeros(M,Lz);

% Initialisation of pi (portfolio)
pi=zeros(M,N+1);

% Initialisation of E (Expectation of Y given the filtration)
E=zeros(M,N+1);

%% Initialisation of Y
Yj=zeros(M,N+1);
% Y at the terminal time
Yj(:,N+1)=max(Sj(:,N+1)-K1,0) - 2*max(Sj(:,N+1)-K2,0);
%% American opt exercise vs continuation
Yj_tilde=zeros(M,N+1);
Yj_tilde(:,N+1)=max(Sj(:,N+1)-K1,0) - 2*max(Sj(:,N+1)-K2,0);
%      Rj=@(x,y) (y+max(x-y,0));

%%
i=N;
while i>1
    % Regression matrix
    for m=1:M
        Mk(m,:)=pol(Sj(m,i+1));
        Mkz(m,:)=polz(Sj(m,i+1));
    end

    % Least squares for pi
    Lzz=chol(Mkz'*Mkz)'; % Cholesky decomposition
    zz=Lzz\Mkz' * ( Yj(:,i+1).*delta_Wj(:,i+1));
    alphaz=Lzz'\zz; % Regression coefficients

    % calculate estimator of pi
    pi(:,i)=Mkz*alphaz/(T/N);

    % Least squares for Y
    Lyy=chol(Mk'*Mk)'; % Cholesky decomposition
    zy=Lyy\Mk' * ( Yj(:,i+1) );
    alpha=Lyy'\zy; % Regression coefficients

    % Calculate estimator for E(i+1)=E[Yi+1 | Fi]
    E(:,i)=Mk*alpha;

    %% Picard iterations to evaluate the estimator of Y
    J=3;
    for j=1:J

```



```

        Yj(:,i)=E(:,i)+(-r*Yj(:,i)-pi(:,i)*lambda ...
        +(R-r)*min(Yj(:,i)-pi(:,i),0))*(T/N);
    end

    %      % Reflections
    %      Yj_tilde(:,i)=Rf((max(Sj(:,i)-K1,0) - 2*max(Sj(:,i)-K2,0)),Yj(:,i));
    %      Yj_tilde(:,i)=max((max(Sj(:,i)-K1,0) - 2*max(Sj(:,i)-K2,0)),Yj(:,i));

    i=i-1;

end

pi=sum( Yj_tilde(:,2).*delta_Wj(:,2) )/(M*(T/N));

%% Picard iterations to evaluate the estimator of Y0
Y=0;
J=10;
for j=1:J
    Y=sum( Yj_tilde(:,2))/M +(-r*Y-pi(:,i)*lambda +(R-r)*min(Y-pi,0))*(T/N);
end

Y0(k)=Y;
mean_Y0=mean(Y0);
std_Y0=std(Y0);

%% 95% confidence interval
upper = mean_Y0 + norminv(0.975) * std_Y0/sqrt(M);
lower = mean_Y0 + norminv(0.025) * std_Y0/sqrt(M);
CI=[lower,upper];

end

for m=1:M
    Yj(m,1)=mean_Y0;
end

[mean_Y0 pi std_Y0 CI]

```

Tables for Result Obtained using the LSMC Method

Table A.1: European call spread using global polynomial bases (The first 3 Laguerre polynomials)

Laguerre polynomials				
M	std	$N = 20$	std	$N = 50$
		$d_y = 2, d_z = 1$ 95% confidence interval		$d_y = 4, d_z = 2$ 95% confidence interval
128	0.3137	[2.8434, 2.9521]	0.3734	[2.8024, 2.9317]
512	0.1622	[2.8207, 2.8488]	0.2022	[2.7794, 2.8144]
2048	0.0959	[2.8047, 2.8130]	0.0992	[2.7810, 2.7896]
8192	0.0508	[2.8182, 2.8204]	0.0474	[2.7672, 2.7693]
32768	0.0276	[2.8078, 2.8084]	0.0227	[2.7681, 2.7686]

Table A.2: European call spread using global polynomial bases (Polynomials of order 0, . . . , 3)

Polynomials of order 0, . . . , 3				
M	std	$N = 20$	std	$N = 50$
		$d_y = 2, d_z = 1$ 95% confidence interval		$d_y = 4, d_z = 2$ 95% confidence interval
128	0.3137	[2.8468, 2.9745]	0.3734	[2.8024, 2.9317]
512	0.1622	[2.8198, 2.8544]	0.2022	[2.7794, 2.8144]
2048	0.0959	[2.8185, 2.8269]	0.0992	[2.7810, 2.7896]
8192	0.0508	[2.8041, 2.8061]	0.0474	[2.7672, 2.7693]
32768	0.0276	[2.8052, 2.8057]	0.0227	[2.7681, 2.7686]

Table A.3: European call spread using local polynomials of order 0

M	$N = 5, \delta = 5$		$N = 10, \delta = 1$	
	(mean, std)	95% confidence interval	(mean, std)	95% confidence interval
128	(2.931727, 0.386886)	[2.864703, 2.998752]	(3.056163, 0.356565)	[2.994391, 3.117935]
512	(2.923739, 0.187648)	[2.907485, 2.939993]	(2.970024, 0.190661)	[2.953509, 2.986539]
2048	(2.919599, 0.103249)	[2.915127, 2.924071]	(2.943109, 0.084924)	[2.939431, 2.946787]
8192	(2.911789, 0.036799)	[2.910992, 2.912586]	(2.938153, 0.051349)	[2.937041, 2.939265]
32768	(2.915952, 0.024432)	[2.915687, 2.916216]	(2.943210, 0.024443)	[2.942945, 2.943474]

Table A.4: American call spread using global polynomial bases (The first 3 Laguerre polynomials)

Laguerre polynomials				
		$N = 20$	$N = 50$	
		$d_y = 2, d_z = 1$	$d_y = 4, d_z = 2$	
M	std	95% confidence interval	std	95% confidence interval
128	0.1938	[5.1617, 5.2288]	0.1029	[5.0226, 5.0583]
512	0.0953	[5.1732, 5.1897]	0.0524	[5.0285, 5.0376]
2048	0.0388	[5.1825, 5.1858]	0.0266	[5.0331, 5.0354]
8192	0.0213	[5.1773, 5.1783]	0.0148	[5.0359, 5.0365]
32768	0.0103	[5.1795, 5.1798]	0.0087	[5.0350, 5.0352]

Table A.5: American call spread using global polynomial bases (Polynomials of order 0, . . . , 3)

Polynomials of order 0, . . . , 3				
		$N = 20$	$N = 50$	
		$d_y = 2, d_z = 1$	$d_y = 4, d_z = 2$	
M	std	95% confidence interval	std	95% confidence interval
128	0.1938	[5.1617, 5.2288]	0.1029	[5.0226, 5.0583]
512	0.0953	[5.1732, 5.1897]	0.0524	[5.0285, 5.0376]
2048	0.0388	[5.1825, 5.1858]	0.0266	[5.0331, 5.0354]
8192	0.0213	[5.1773, 5.1783]	0.0148	[5.0359, 5.0365]
32768	0.0103	[5.1795, 5.1798]	0.0087	[5.0350, 5.0352]